

Ingegneria del Software II – Modulo di Software Testing

Report di Progetto

Analisi e Qualificazione delle Suite di Test

Progetto: Apache Syncope

Studente: Leonardo Polidori (0357314)

Repository GitHub: <https://github.com/Polidori1999/syncope>

1 Analisi e Testing: DefaultJobManager

L'attività di testing su `DefaultJobManager` si concentra inizialmente sul metodo `execute(SchedTask task, OffsetDateTime startAt, String executor, boolean dryRun, Map<String, Object> jobData)` punto di ingresso pubblico per l'esecuzione di un task schedulato. Il metodo riceve il task, l'eventuale istante di avvio, l'esecutore, il flag `dryRun` e una mappa di dati aggiuntivi; consente quindi di osservare sia la gestione degli input sia l'interazione con i collaboratori responsabili della creazione e schedulazione del job.

Dopo `execute(...)`, l'analisi viene estesa a `load(String domain)`, relativo all'inizializzazione dei job già presenti nel sistema. I due metodi coprono responsabilità diverse della classe: schedulazione puntuale di un task e caricamento dei job in fase di avvio.

1.1 Fase 1: Progettazione e Generazione dei Test

La prima fase ha riguardato la progettazione di una suite manuale per la classe originale C_0 , adottando un approccio funzionale *black-box*. Le categorie sono state derivate dalla firma dei metodi, dalla documentazione disponibile e dal significato dei parametri nel dominio applicativo.

1.1.1 Test Manuali (Category Partition) - Metodo `execute`

La progettazione manuale dei test è stata condotta applicando il metodo del *Category Partition* al dominio di input del metodo:

```
execute(SchedTask task, OffsetDateTime startAt, String executor,  
        boolean dryRun, Map<String, Object> jobData)
```

La partizione è stata costruita in due passaggi. In una prima fase è stata definita una partizione grezza, distinguendo per i parametri oggetto il caso di valore assente, cioè `null`, dal caso di valore presente, cioè non `null`. Per il parametro booleano `dryRun`, invece, sono stati considerati direttamente i due valori possibili: `true` e `false`.

La partizione è stata poi raffinata, considerando la firma pubblica del metodo, le interfacce esposte dai tipi coinvolti e il significato dei parametri nel dominio considerato, in classi di equivalenza più specifiche: task nullo, task attivo, task inattivo e task privo di *job delegate*; `startAt` nullo, passato, presente o futuro; `executor` nullo, vuoto o valorizzato; `jobData` nulla, vuota o non vuota.

La raffinazione distingue validità sintattica e validità semantica. Ad esempio, `executor = ""` è sintatticamente accettabile perché rispetta il tipo `String`, ma non identifica un esecutore reale. Analogamente, `startAt` nel passato è un `OffsetDateTime` valido, ma è stato considerato inizialmente problematico rispetto alla semantica di una schedulazione. Per `jobData`, il valore `null` rappresenta invece l'assenza della mappa dati e richiede una gestione esplicita.

I rappresentanti concreti sono stati scelti per tradurre le classi di equivalenza in valori eseguibili nei test JUnit. Per i valori temporali è stata applicata una logica vicina alla *Boundary Value Analysis*, scegliendo un istante passato, uno presente e uno futuro. Per gli altri parametri sono stati usati rappresentanti semplici ma distintivi, come `null`, `""`, `"admin"`, `Map.of()` e una mappa non vuota. Poiché `SchedTask` è un'interfaccia, i rappresentanti del task sono stati realizzati tramite mock, configurati esclusivamente per modellare gli stati rilevanti individuati nella partizione. I rappresentanti concreti sono riportati nella Tabella 1.

Per evitare l'esplosione combinatoria, non è stato generato il prodotto cartesiano completo. I casi validi combinano scelte valide, mentre i casi invalidi introducono una sola anomalia alla volta rispetto a uno scenario nominale. In questo modo ogni classe di equivalenza significativa viene esercitata almeno una volta, mantenendo però la suite leggibile e interpretabile. L'applicazione di questi criteri ha portato alla definizione di 9 test frame astratti, indicati come TF1–TF9, riportati nella Tabella 2.

Gli oracoli sono stati definiti prima dell'esecuzione dei test, in termini di comportamento osservabile: schedulazione del job, assenza di schedulazione, creazione del `JobExecutionContext`, propagazione dei dati o rifiuto dell'input. Nei casi nominali l'oracolo prevede la preparazione del contesto e l'invocazione dello scheduler; con task inattivo prevede invece l'assenza di creazione e schedulazione del job. Per input chiaramente non validi, come task nullo o privo di *job delegate*, l'oracolo prevede il rifiuto dell'esecuzione.

Alcuni oracoli sono stati formulati come ipotesi iniziali, perché riguardano valori semanticamente sospetti ma non vietati in modo esplicito dalla documentazione. In particolare, `startAt` nel passato, `executor` nullo o vuoto e `jobData = null` sono stati trattati come casi da verificare sperimentalmente. La legenda degli oracoli è riportata nella Tabella 3.

1.1.2 Test Manuali di Integrazione - Metodo `load`

Dopo i test manuali sul metodo `execute(...)`, l'analisi è stata estesa al metodo pubblico:

```
load(String domain)
```

Come accennato prima il metodo `load(String domain)` non esegue una semplice computazione locale, ma inizializza i job associati a un dominio. Il test non mira quindi a verificare un valore di ritorno, ma il corretto avvio del flusso di inizializzazione coordinato da `DefaultJobManager`.

La classe `DefaultJobManagerLoadBBIT` è stata mantenuta come test di integrazione leggero, più precisamente come *component-level integration test*. Non vengono avviati l'intero ambiente Syncope, il database reale, il container Spring reale o uno scheduler reale; tuttavia, il componente coordinatore `DefaultJobManager` rimane reale e collabora con dipendenze sostituite da mock o stub controllati. Il test si colloca quindi tra unit test puro ed end-to-end test, secondo una strategia di integrazione controllata top-down e non di tipo *Big Bang*. L'obiettivo di questo approccio non è validare la logica interna del singolo componente, bensì verificare l'avvenuta comunicazione e il corretto passaggio di parametri tra il coordinatore e i livelli inferiori simulati. In questo contesto, l'uso di `verify()` consente di controllare che le interazioni attese e il passaggio dei parametri avvengano nell'ambiente di test configurato.

La tecnica di selezione dei casi resta comunque *black-box*. La *Category Partition* è stata applicata al parametro esplicito `domain`, senza derivare i casi dai rami interni dell'implementazione. La documentazione non specifica vincoli forti sui valori ammessi per il dominio; per questo motivo la partizione distingue un caso nominale, rappresentato da una stringa significativa come "test-domain", e due casi limite: `domain = null` e `domain = ""`. Questi ultimi non sono stati classificati come input certamente invalidi, ma come valori da caratterizzare sperimentalmente. I rappresentanti concreti sono riportati nella Tabella 4.

In questo test il solo parametro variato è `domain`; il resto del fixture rappresenta l'ambiente minimo necessario per eseguire `load(...)` fuori dal runtime applicativo reale. I DAO sono stati configurati per restituire liste vuote di task e report (perché l'obiettivo è verificare il completamento del flusso di inizializzazione rispetto ai valori del parametro), mentre una `DefaultListableBeanFactory` mockata è stata registrata tramite `ApplicationContextProvider` per fornire dipendenze ambientali come `SecurityProperties`. Questa configurazione non definisce l'oracolo, ma rende eseguibile il flusso di inizializzazione in modo controllato.

L'applicazione di questi criteri ha portato alla definizione di tre test frame, riportati nella Tabella 5. Per il dominio nominale, l'oracolo iniziale prevede il completamento del metodo senza eccezioni inattese. Per `domain = null` e `domain = ""`, invece, l'oracolo iniziale ipotizza il rifiuto dell'input tramite eccezione. Tali oracoli sono stati trattati come predizioni da verificare, perché la documentazione del metodo non specifica esplicitamente che questi valori debbano essere rifiutati. La legenda degli oracoli è riportata nella Tabella 6.

I test sono stati collocati nella classe `DefaultJobManagerLoadBBIT` e delegati al plugin Maven Failsafe. L'esecuzione materiale avviene durante la fase di *integration-test*, mentre la validazione dei risultati è demandata alla fase di *verify*. Questo approccio permette di separarli rigorosamente dai test unitari, gestiti invece dal plugin Surefire.

1.1.3 Esecuzione dei test manuali Black-Box

La suite manuale Black-Box relativa a `execute(...)` è stata eseguita nella classe `DefaultJobManagerBBTest`. L'obiettivo era verificare la classe originale rispetto ai test frame progettati tramite *Category Partition*, dis-

tinguendo i casi confermati dagli oracoli iniziali non confermati. L'esecuzione Maven ha prodotto:

```
Tests run: 13, Failures: 0, Errors: 0, Skipped: 4
```

La build è terminata con esito positivo. I test attivi risultano quindi coerenti con gli oracoli mantenuti come eseguibili; i quattro test saltati non rappresentano instabilità, ma documentano ipotesi iniziali non confermate.

In particolare, i casi con `startAt` nel passato e con `executor` nullo o vuoto non hanno prodotto il rifiuto ipotizzato in fase di progettazione. L'implementazione procede invece con la preparazione del job e la delega allo scheduler. Poiché la documentazione disponibile non impone esplicitamente il rifiuto di tali valori, questi esiti non sono stati classificati come bug certi: gli oracoli iniziali sono stati conservati come test disabilitati, mentre la suite attiva documenta il comportamento osservato.

Il caso `jobData = null` è stato trattato diversamente. L'oracolo iniziale ammetteva una gestione protetta dell'input nullo oppure un rifiuto controllato; l'implementazione solleva invece una `NullPointerException`. Il comportamento è stato quindi documentato come non robusto e come possibile difetto di gestione dell'input nullo. Non è possibile escludere che il controllo sia previsto a monte, cioè nel chiamante, e che il metodo assuma implicitamente una mappa non nulla. Anche qui la build finale non viene lasciata fallire: il comportamento corrente è fissato tramite `characterization test`, mentre l'oracolo ideale resta documentato come test disabilitato.

Quindi, una build verde non dimostra l'assenza di difetti, ma indica coerenza tra test attivi e oracoli scelti dopo l'analisi. I test disabilitati tracciano invece ipotesi iniziali che non hanno trovato conferma sperimentale o che richiederebbero una specifica più forte per essere considerate bug certi.

Per il metodo `load(...)`, l'esecuzione della classe `DefaultJobManagerLoadBBIT` ha prodotto il seguente risultato:

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 2
```

Il numero riportato da Maven si riferisce ai metodi JUnit presenti nella classe, non direttamente ai valori della partizione. I tre valori di `domain` ("`test-domain`", `null` e `""`) sono infatti esercitati in un unico test attivo, `loadShouldCompleteInitializationForAcceptedDomainValues()`, per evitare duplicazione del fixture e ridurre test smell.

Il test attivo configura lo stesso ambiente controllato per ciascun valore: DAO con liste vuote di task e report, `DefaultListableBeanFactory` mockata per fornire `SecurityProperties` e invocazione di `load(...)` tramite `assertDoesNotThrow`. Tra le iterazioni i mock principali vengono resettati per evitare interferenze.

Il comportamento osservato è che `load(...)` completa senza eccezioni non solo per il dominio nominale, ma anche per `null` e `""`. Questo smentisce gli oracoli iniziali sui due casi limite, che ipotizzavano un rifiuto tramite eccezione. Poiché la documentazione non richiede esplicitamente tale rifiuto, la discrepanza non è stata classificata automaticamente come bug. I due test saltati conservano quindi traccia dell'aspettativa iniziale, mentre il test attivo documenta il comportamento corrente dell'implementazione.

1.2 Analisi JaCoCo dopo i test Black-Box

Dopo l'esecuzione delle suite Black-Box su `execute(...)` e `load(...)`, JaCoCo è stato utilizzato come misura di adeguatezza a posteriori. Il report JaCoCo è stato usato per osservare quali parti di `DefaultJobManager` fossero state effettivamente esercitate.

L'overview della classe mostra una copertura parziale, come riportato nelle Figure 1 e 2. Questo risultato è coerente con il perimetro dei test: la suite Black-Box non mira a coprire tutta la classe, ma si concentra sui metodi `execute(...)` e `load(...)`. Restano quindi fuori altri comportamenti, come registrazione diretta di task e report, `unregister`, `unload` e rami specifici legati al dominio master.

Nel dettaglio, `execute(...)` risulta attraversato come punto di ingresso pubblico. La logica delegata viene però esercitata solo nei rami coerenti con i test progettati: task attivo, task inattivo, task privo di *job delegate* e propagazione del contesto del job, come mostrato nella Figura 3. Restano scoperti rami specifici, ad esempio quelli relativi a `PullTask`, `PushTask`, registrazione senza data esplicita e schedulazione tramite `CronTrigger`. Tali rami non indicano un fallimento della suite Black-Box, ma possibili candidati per un successivo affinamento `coverage-aware`.

Per `load(String domain)`, gli screenshot mostrano che viene attraversata la parte iniziale del metodo: il flusso entra nell'inizializzazione, interroga i DAO e recupera le liste di task e report, come evidenziato nella Figura 4. Poiché il fixture Black-Box configura i DAO per restituire liste vuote, i corpi dei cicli che registrano task e report non vengono esercitati. Questa scelta è coerente con l'obiettivo del test Black-Box: verificare il completamento del flusso rispetto ai valori di `domain`, non la registrazione effettiva di entità specifiche.

La parte finale di `load(...)` rimane quindi in larga parte scoperta, come mostrato nella Figura 5. Anche in questo caso il dato non è stato interpretato come errore della suite, ma come indicazione diagnostica: i rami relativi a `task`, `report`, `job` di sistema e dominio master richiedono una progettazione strutturale dedicata.

Nel complesso, JaCoCo conferma il perimetro effettivo della suite Black-Box. I test coprono i punti di ingresso e i comportamenti coerenti con la partizione iniziale; i rami scoperti diventano invece input per la successiva fase di raffinamento strutturale.

1.3 Fase 2: Raffinamento strutturale guidato da JaCoCo

Dopo l'esecuzione delle suite Black-Box, JaCoCo è stato usato come strumento di *adequacy* per individuare porzioni significative di codice non ancora esercitate. La coverage non è stata interpretata come obiettivo numerico da massimizzare, ma come supporto diagnostico: un ramo rosso segnala codice non attraversato, ma non implica automaticamente l'aggiunta di un test.

Il criterio adottato è stato selettivo. Sono stati aggiunti test solo quando il ramo scoperto rappresentava un comportamento normale, plausibile o architetturealmente rilevante di `DefaultJobManager`, oppure quando permetteva di verificare una collaborazione significativa tra il componente e i suoi collaboratori. Non sono stati invece introdotti test mirati esclusivamente ad aumentare la percentuale di coverage.

1.3.1 Raffinamento del flusso `execute(...)`

Per `execute(...)`, JaCoCo ha mostrato che la suite Black-Box copriva il ramo in cui un job viene schedulato a un istante specifico, ma non quello in cui il job viene solo registrato. Questo secondo comportamento si verifica quando `startAt` è nullo e non è presente una cron expression.

La scelta di coprirlo è stata motivata dal fatto che registrazione e schedulazione a un `Instant` sono due interazioni diverse con lo scheduler. Inoltre, `startAt = null` era già una classe di equivalenza prevista nella progettazione iniziale, ma nei test Black-Box era stata combinata con un task inattivo, arrestando il flusso prima di `registerJob(...)`.

È stato quindi aggiunto un test con task valido e attivo, `startAt = null`, `executor = "admin"` e `jobData` vuota. In questo caso l'oracolo non riguarda un valore di ritorno, ma l'interazione attesa con lo scheduler: il metodo deve predisporre il job e il relativo contesto di esecuzione, registrando il job senza schedularlo a un `Instant` specifico. Per questo motivo il test verifica l'invocazione di `register(...)` e l'assenza di invocazione di `schedule(..., Instant)`. Il test è strutturale nella motivazione, ma rimane coerente con il dominio perché esercita una combinazione plausibile già emersa dalla partizione.

1.3.2 Raffinamento del flusso `load(...)`

Per `load(...)`, i test Black-Box configuravano i DAO per restituire liste vuote. Questa scelta era coerente con l'obiettivo iniziale: verificare il completamento dell'inizializzazione rispetto ai valori di `domain`, senza introdurre nella prima fase la registrazione effettiva di task e report.

JaCoCo ha però mostrato che i corpi dei cicli di registrazione di task e report non venivano attraversati. Questi rami sono stati considerati rilevanti perché rappresentano il comportamento principale di `load(...)` quando nel dominio esistono job da inizializzare.

Sono stati quindi aggiunti due test con DAO valorizzati: uno con `SchedTask` attivo e cron expression, e uno con `Report` attivo e cron expression. Gli oracoli verificano la creazione del job, l'impostazione del `JobExecutionContext` e la schedulazione tramite `CronTrigger`.

Un ulteriore ramo scoperto riguardava il dominio master, non coperto dai valori iniziali della partizione su `domain`: `"test-domain"`, `null` e `""`. La copertura di questo ramo è stata ritenuta utile perché il dominio master attiva logiche di sistema distinte dal caricamento ordinario di task e report.

Sono stati quindi introdotti test mirati sul dominio master: assenza di una cron expression significativa per il notification job e presenza di una cron expression configurata. In questo modo vengono esercitati i rami relativi a `NotificationJob` e `SystemLoadReporterJob`, mantenendo un ambiente controllato tramite mock.

Questi casi non sostituiscono la progettazione Black-Box su `domain`, ma costituiscono un raffinamento white-box.

1.3.3 Risultato del raffinamento e decisione di stop

Dopo il raffinamento, la copertura complessiva di `DefaultJobManager` aumenta al 64% di instruction coverage e al 77% di branch coverage, come mostrato nella Figura 6. Il dettaglio dei metodi evidenzia un aumento

sui flussi principali di `load(...)`, `register(...)` e `registerJob(...)`, riportati nelle Figure 7 e 8. Alcuni rami restano non coperti. In particolare, non sono stati aggiunti test per tutti i blocchi `catch` nei flussi di caricamento di task, report e job di sistema. Per attraversarli sarebbe stato necessario configurare artificialmente i mock affinché fallissero operazioni interne, come creazione del job, registrazione sullo scheduler o accesso ai dati dei DAO. Questi scenari non sono stati esclusi perché impossibili da testare, ma perché avrebbero verificato soprattutto la reazione a eccezioni prodotte forzatamente dai mock configurati, più che un requisito o una collaborazione applicativa significativa. La decisione di stop è quindi motivata da un criterio costo/beneficio: sono stati coperti i rami normali, plausibili o architetturealmente rilevanti, mentre sono stati esclusi quelli costruiti solo per aumentare la percentuale di coverage.

1.4 Fase 3: Adeguatezza semantica tramite Mutation Testing

Dopo JaCoCo, la suite è stata analizzata con PIT per valutare la forza degli oracoli.

Anche PIT è stato usato come strumento diagnostico di *adequacy*, non come metrica da massimizzare meccanicamente. Un mutante sopravvissuto può indicare un oracolo debole, un ramo non coperto, un comportamento fuori scope o un caso il cui costo di modellazione è sproporzionato rispetto al beneficio atteso.

1.4.1 Baseline iniziale

La prima esecuzione di PIT sulla classe `DefaultJobManager` ha prodotto una baseline pari al 59% di *Mutation Coverage*, con 32 mutanti uccisi su 54. La panoramica iniziale è riportata nella Figura 9.

Il report ha evidenziato due categorie di mutanti residui: 17 `NO_COVERAGE`, generati in porzioni di codice non attraversate, e 5 `SURVIVED`, raggiunti dai test ma non intercettati dagli oracoli. La ripartizione è riportata nella Figura 10. La distinzione è importante: i primi indicano soprattutto un limite di copertura, i secondi segnalano una possibile debolezza semantica degli oracoli.

1.4.2 Interventi di raffinamento

Il raffinamento è stato applicato solo ai mutanti ancora coerenti con lo scope della suite. Un primo intervento ha riguardato un mutante `NO_COVERAGE` nel metodo `getOrder()`, generato dal `PrimitiveReturnsMutator`: è stato introdotto un test che verifica esplicitamente il valore restituito, riducendo i mutanti non coperti da 17 a 16.

Un secondo intervento ha riguardato un mutante `SURVIVED` associato a una chiamata con side-effect. Il report indicava che un mutante del `VoidMethodCallMutator` sopravviveva perché i test non verificavano esplicitamente l'interazione con un collaboratore mockato. L'oracolo è stato quindi rafforzato con verifiche `verify()` mirate, controllando non solo il completamento del flusso ma anche l'effettivo side-effect osservabile.

Dopo il raffinamento, PIT mostra un miglioramento complessivo: la *Mutation Coverage* sale al 65%, con 35 mutanti uccisi su 54. Il riepilogo finale è riportato nella Figura 11.

In particolare, i mutanti `SURVIVED` passano da 5 a 3. L'aumento della *Test Strength* dall'86% al 92% è il segnale più rilevante: una volta raggiunto il codice mutato, gli oracoli risultano più efficaci nel distinguere il comportamento corretto da quello alterato.

Il processo è stato arrestato sulla base di un criterio costo-beneficio. I mutanti residui sono stati documentati, ma non inseguiti in quanto la loro uccisione avrebbe richiesto test artificiali, fortemente dipendenti dall'implementazione o fuori dal perimetro funzionale considerato. Il risultato finale non indica perfezione della suite, ma una buona adeguatezza semantica rispetto agli obiettivi del lavoro.

1.5 Generazione automatica con Randoop

È stata valutata la generazione automatica tramite Randoop, con l'obiettivo di osservare quale contributo potesse fornire una tecnica randomica e di confrontarne i risultati con quelli delle suite manuali.

Randoop è un generatore feedback-directed: costruisce sequenze di chiamate a partire dalle API disponibili, osserva quali sequenze terminano correttamente e le trasforma in regression test che fissano il comportamento osservato come oracolo.

Con un time budget di 60 secondi, Randoop ha generato 8 test compilabili. Tuttavia, la suite autogenerata ha prodotto 0% di *Instruction Coverage*, 0% di *Branch Coverage* e nessun contributo in PIT: tutti i 54 mutanti sono rimasti `NO_COVERAGE`. L'evidenza JaCoCo è riportata nella Figura 12.

Il risultato è coerente con la natura del SUT: `DefaultJobManager` richiede un fixture di dominio con DAO, scheduler, bean factory, job e proprietà di sicurezza. Randoop genera sequenze dalle API disponibili, ma non costruisce automaticamente un ambiente Syncope valido. Di conseguenza, gli 8 test generati non aumentano né la coverage né il mutation score sulla classe target; integrarli nella suite finale avrebbe quindi aggiunto costo di mantenimento senza fornire evidenza aggiuntiva sui comportamenti rilevanti.

1.6 Generazione tramite LLM

Successivamente è stato condotto un esperimento tramite LLM, usando Copilot Think Deeper. L'obiettivo non era sostituire la suite manuale, ma valutare se una strategia di prompting guidata potesse produrre test compilabili, pertinenti e utili rispetto alle metriche di adequacy.

Sono state considerate tre configurazioni. Nel caso zero-shot senza codice sorgente, il modello ha generato test generici, prevalentemente basati su reflection, assumendo anche l'esistenza di un costruttore pubblico senza argomenti non presente nella classe. Nel caso zero-shot con source code, i test erano più vicini al SUT e introducevano Mockito, ma la suite non compilava per errori nelle API del progetto, fixture incompleto e gestione della bean factory; un esempio è riportato nella Figura 13.

La configurazione finale ha usato una strategia *Guided Tree-of-Thought*. Il prompt, riportato nel Listato 1, richiedeva al modello di analizzare prima i metodi pubblici e protetti, distinguere casi normali, limite ed eccezionali, e solo dopo generare una classe JUnit 4.

Il primo output Guided Tree-of-Thought era più pertinente degli zero-shot, perché esercitava metodi come `isRunning(...)`, `registerJob(...)`, `execute(...)`, `unregister(...)` e `getOrder()`. Tuttavia, sono state necessarie quattro iterazioni di fixing conversazionale: correzione del tipo `Job`, adeguamento alle API Mockito, completamento dello stub di `TaskUtilsFactory` e correzione del flusso legato a `jobStatusDAO.lock(...)`. È stata poi effettuata una minima correzione manuale per rimuovere stub inutilizzati segnalati da Mockito, senza modificare gli oracoli.

La suite LLM finale contiene 12 test eseguibili, con 0 failure e 0 errori. Questo mostra che il Guided Tree-of-Thought, accompagnato da validazione Maven e fixing mirati, può produrre una suite tecnicamente utilizzabile; tuttavia, il risultato non è completamente autonomo, perché richiede revisione da parte del tester.

La suite è stata valutata separatamente con JaCoCo e PIT. Dal punto di vista strutturale ha raggiunto il 33% di *Instruction Coverage* e il 50% di *Branch Coverage*, come mostrato nelle Figure 14 e 15. Il risultato è nettamente migliore rispetto a Randoop, ma resta inferiore alla suite manuale raffinata: l'LLM copre soprattutto comportamenti circoscritti e locali, mentre fatica sui flussi di inizializzazione più complessi, in particolare `load(...)`, che richiedono fixture coerenti con DAO, bean factory, scheduler e proprietà di sicurezza.

Dal punto di vista semantico, PIT ha registrato 23 mutanti uccisi su 54, con *Mutation Coverage* pari al 43% e *Test Strength* pari all'88%, come riportato nella Figura 16. Il dettaglio dei mutanti è mostrato nella Figura 17.

È stata infine valutata anche la configurazione combinata composta dalla suite manuale finale e dalla suite LLM Guided Tree. In questa configurazione, `DefaultJobManager` raggiunge il 69% di line coverage e l'85% di branch coverage, come mostrato nelle Figure 18 e 19. Anche PIT mostra un incremento rispetto alla sola suite manuale: i mutanti uccisi salgono a 40 su 54, con *Mutation Coverage* pari al 74% e *Test Strength* pari al 95%, come riportato nella Figura 20.

Questo risultato indica che, pur non sostituendo la progettazione manuale, la suite LLM fornisce un contributo incrementale: alcuni test generati raggiungono porzioni della classe non esercitate dalla suite manuale e permettono di uccidere mutanti aggiuntivi. La valutazione combinata conferma quindi il ruolo dell'LLM come supporto complementare, utile ad ampliare la copertura e la forza semantica della suite, ma non sufficiente da solo a sostituire la progettazione manuale.

1.7 Limitazione nell'uso di EvoSuite su DefaultJobManager

La generazione tramite EvoSuite non è stata applicabile a `DefaultJobManager` per un vincolo tecnico di compatibilità del bytecode. EvoSuite genera test automaticamente a partire dal bytecode della classe target: strumentata il codice, esplora sequenze di input con algoritmi di ricerca e produce test JUnit con asserzioni derivate dal comportamento osservato durante l'esecuzione. Durante i tentativi di esecuzione, EvoSuite ha terminato con l'errore `Unsupported class file major version`, dovuto al fatto che il tool richiede classi compatibili con Java 8, cioè major version 52, mentre le versioni disponibili di `DefaultJobManager` risultano compilate con bytecode più recente.

È stata valutata anche la possibilità di usare una versione legacy della classe, ma non è stata individuata una variante utilizzabile con bytecode 52. Di conseguenza, non è stato possibile generare una suite EvoSuite compilabile e integrabile nel progetto.

Per questo motivo, nelle matrici sperimentali EvoSuite è indicato come N/A: non si tratta di un fallimento dei test generati, ma di una generazione non tecnicamente applicabile al target considerato.

1.8 Generazione delle varianti refactorizzate

Dopo il raffinamento della suite di test su C_0 , l'analisi è stata estesa alla generazione automatica di varianti refactorizzate di `DefaultJobManager`. La classe originale è stata usata come baseline C_0 ; a partire da essa sono state generate quattro varianti, C_1 – C_4 . Nel codice Java il nome della classe resta sempre `DefaultJobManager`, mentre le etichette distinguono solo le configurazioni sperimentali. La configurazione degli input forniti all'LLM è riportata nella Tabella 7.

1.9 Validazione delle varianti tramite suite di regressione

Dopo la generazione, ciascuna variante è stata copiata temporaneamente nel path originale della classe ed eseguita con le suite sviluppate su C_0 . L'obiettivo non era verificare la sola compilabilità, già controllata separatamente, ma stabilire se la variante potesse sostituire la classe originale preservando il comportamento coperto dagli oracoli. La matrice degli output osservati è riportata nella Tabella 8.

Il risultato mostra che C_0 e C_1 superano l'intera suite disponibile, mentre C_2 , C_3 e C_4 falliscono pur essendo compilabili. La causa è una differenza comportamentale intercettata dal test sul caso `jobData = null` (`executeWithNullJobDataCurrentlyThrowsNullPointerException`). In C_0 , questo input termina con una `NullPointerException`; tale comportamento non è desiderabile in assoluto, ma è stato caratterizzato come comportamento osservato della baseline. Nelle varianti C_2 , C_3 e C_4 , invece, il refactoring introduce una gestione difensiva del valore nullo, evitando o sostituendo la copia della mappa. Il confronto è riportato nel Listato 2.

Il punto non è stabilire se la gestione difensiva di `jobData = null` sia migliore o peggiore del comportamento originale. La matrice Pass/Fail verifica se una variante riproduce il comportamento osservato in C_0 sui casi coperti dalla suite. In C_0 , il caso nullo è stato caratterizzato tramite una `NullPointerException`; nelle varianti C_2 , C_3 e C_4 tale eccezione non viene più sollevata. Il comportamento coperto dal test è quindi cambiato e le tre varianti vengono respinte come replacement della baseline nel perimetro osservato.

Il caso è metodologicamente rilevante perché C_1 , pur generata con meno contesto di test, mantiene il comportamento originale su `jobData == null`. Le varianti successive sembrano invece interpretare il valore nullo come un caso da gestire in modo più robusto. Questo mostra che fornire più test in input all'LLM non garantisce automaticamente maggiore fedeltà semantica: un refactoring automatico può compilare e produrre codice apparentemente più difensivo, ma resta accettabile solo se preserva gli oracoli costruiti su C_0 .

1.10 Confronto dei test automatici generati sulle varianti

Infine, è stato condotto il confronto tra i test automatici generati per la classe originale C_0 e quelli generati ex novo per le varianti refactorizzate C_1 – C_4 .

Questa fase ha uno scopo diverso rispetto alla validazione tramite suite di regressione. In questa analisi, infatti, per ogni variante viene generata una nuova suite automatica e se ne misura l'adeguatezza rispetto alla variante attiva. Il confronto riguarda quindi l'efficacia dei generatori automatici, non la sostituibilità comportamentale delle varianti.

Le suite considerate sono Randoop (RND) e LLM Guided Tree-of-Thought (LLM); EvoSuite è indicato come N/A per l'incompatibilità di bytecode già discussa. Le metriche scelte sono la *Line Coverage* di JaCoCo, come indicatore strutturale, e il *Mutation Score* di PIT, come indicatore semantico. Per ogni variante è stato calcolato il delta rispetto alla baseline della stessa famiglia di test su C_0 :

$$\Delta Coverage = Coverage(C_X) - Coverage(C_0)$$

$$\Delta Mutation = MutationScore(C_X) - MutationScore(C_0)$$

La matrice completa è riportata nella Tabella 9; i dettagli delle suite LLM sono riportati nella Tabella 10. Il primo risultato evidente riguarda Randoop: per tutte le versioni, da C_0 a C_4 , coverage e mutation score restano pari a 0%. Le suite LLM, invece, raggiungono porzioni reali della classe, ma rispetto alla baseline C_0 mostrano valori inferiori sia in line coverage sia in mutation score. C_2 è la variante più vicina a C_0 dal punto di vista semantico, con un delta mutation pari a -1% , mentre C_4 registra il calo più marcato, pari a -8% .

Questi risultati vanno letti insieme alla matrice di regressione. C_2 , C_3 e C_4 avevano già mostrato una deviazione rispetto al comportamento osservato di C_0 , dovuta alla diversa gestione di `jobData == null`. I loro valori quantitativi sono quindi utili per valutare l'efficacia delle suite generate, ma non bastano a promuovere le varianti come replacement corretti di C_0 . Al contrario, C_1 preserva il comportamento coperto dalla regressione, pur non ottenendo il miglior delta nella Tabella 9.

È stata infine osservata la qualità statica dei test LLM generati attraverso l'analisi dei code smells. Questa analisi non sostituisce JaCoCo e PIT, ma fornisce un'indicazione sulla manutenibilità del codice di test prodotto automaticamente.

Nel conteggio sono state escluse le segnalazioni dovute al mismatch tra path del file e package Java, perché dipendono dalla scelta sperimentale di archiviare i test in `refactoring-variants/.../generated-tests`, fuori dal normale source tree Maven. Le issue considerate riguardano invece naming, duplicazione di literal e altri aspetti di manutenibilità; gli screenshot sono riportati nelle Figure 21–25 e il riepilogo nella Tabella 11.

Una suite con meno smell è più pulita e meno costosa da mantenere, ma non è automaticamente più efficace né rende corretta la variante su cui è stata generata.

Il caso di C_2 è indicativo: pochi smell nella suite LLM non implicano automaticamente una variante più affidabile, perché resta presente una deviazione rispetto al comportamento osservato di C_0 . La valutazione deve quindi rimanere multidimensionale, combinando efficacia dei test, mutation score, manutenibilità della suite e compatibilità con gli oracoli della baseline.

2 Analisi e Testing: AuthDataAccessor

La seconda classe selezionata è `AuthDataAccessor`, componente del sottosistema di sicurezza di Apache Syncope.

L'analisi si concentra sui due metodi pubblici principali:

```
authenticate(String domain, Authentication authentication)
getAuthorities(String username, String delegationKey)
```

Il primo metodo valuta una richiesta di autenticazione a partire dal dominio e dal token Spring Security ricevuto in input; il secondo ricostruisce le authorities associate a un utente, eventualmente in un contesto di delega.

La progettazione iniziale segue un approccio prevalentemente *black-box*: le categorie di input derivano dalla firma dei metodi, dalla documentazione e dal significato dei parametri nel dominio di sicurezza.

2.1 Fase 1: Progettazione dei test manuali Black-Box - Metodo `authenticate`

La progettazione su `AuthDataAccessor` applica lo stesso schema di *Category Partition* già discusso, adattandolo al metodo:

```
authenticate(String domain, Authentication authentication)
```

La partizione è stata costruita in due passaggi, come per `execute(...)`. In una prima fase è stata definita una partizione grezza, distinguendo per i parametri oggetto il caso di valore assente, cioè `null`, dal caso di valore presente, cioè `non null`.

La partizione è stata poi raffinata considerando la firma pubblica del metodo, i tipi coinvolti e il significato dei parametri nel flusso di autenticazione. Per `domain` sono stati distinti dominio valorizzato, dominio vuoto e dominio nullo. Per `authentication` sono stati distinti token assente, token completo, token privo di principal e token privo di credenziali. I rappresentanti concreti sono riportati nella Tabella 12.

Anche qui viene mantenuta la distinzione tra validità sintattica e semantica: `domain = ""` è passabile al metodo, ma può essere problematico nel dominio di autenticazione; allo stesso modo, un token senza principal o credenziali è costruibile, ma incompleto rispetto all'identificazione dell'utente.

Per evitare l'esplosione combinatoria non è stato generato il prodotto cartesiano completo. È stato usato uno scenario nominale, composto da dominio pieno, token presente, principal e credenziali valorizzati e utente riconosciuto nel contesto di test. Da esso sono stati derivati i casi limite e critici: dominio vuoto, dominio nullo, token nullo, credenziali nulle e principal nullo. I test frame selezionati sono riportati nella Tabella 13.

Gli oracoli sono stati definiti prima dell'esecuzione dei test, in termini di comportamento osservabile: autenticazione positiva, assenza di utente autenticato oppure rifiuto controllato dell'input. Per i casi `authentication = null` e credenziali nulle, l'oracolo *black-box* iniziale prevede un fallimento esplicito dell'autenticazione, senza considerare desiderabile una dereferenziazione non controllata. La legenda degli oracoli è riportata nella Tabella 14.

2.2 Fase 1: Progettazione dei test manuali Black-Box - Metodo `getAuthorities`

La progettazione manuale è stata estesa al secondo metodo pubblico selezionato:

```
getAuthorities(String username, String delegationKey)
```

Il metodo è rilevante perché ricostruisce le authorities associate a un utente, distinguendo tra richiesta diretta e richiesta in contesto di delega.

La partizione grezza considera entrambi i parametri come stringhe. Per `username` sono stati distinti stringa piena, stringa vuota e valore `null`; la stessa scomposizione è stata applicata a `delegationKey`. La raffinazione attribuisce poi significato di dominio a queste scelte: uno `username` pieno rappresenta un possibile utente, mentre `username` vuoto o nullo indicano un identificatore assente o privo di contenuto. Per `delegationKey`, il valore `null` rappresenta assenza di delega, una stringa piena rappresenta una richiesta delegata, e la stringa vuota una chiave formalmente passabile ma semanticamente non informativa.

I rappresentanti concreti sono riportati nella Tabella 15.

Anche in questo caso non è stato generato il prodotto cartesiano completo: sono stati selezionati cinque frame, variando una sola dimensione alla volta rispetto allo scenario nominale. Le combinazioni con doppia anomalia, ad esempio `username` vuoto e `delegation key` piena, sono state escluse perché avrebbero sommato cause già esercitate separatamente. I test frame selezionati sono riportati nella Tabella 16.

Per la richiesta diretta con `delegationKey = null`, l'oracolo prevede la restituzione delle authorities associate all'utente configurato nel fixture. Per la richiesta con `delegationKey` piena, l'oracolo prevede la restituzione di un set di authorities nel contesto della delega indicata.

Per i casi semanticamente problematici, cioè `delegationKey = ""`, `username = ""` e `username = null`, l'oracolo black-box iniziale non impone una specifica eccezione Java. L'attesa è che il metodo non restituisca un set valido di authorities e segnali in modo controllato l'impossibilità di risolvere l'utente o la delega richiesta. La legenda degli oracoli è riportata nella Tabella 17.

2.3 Esecuzione dei test manuali Black-Box

La suite Black-Box iniziale è stata implementata nella classe `AuthDataAccessorBBTest`. Dal punto di vista JUnit, la classe contiene nove metodi di test; dal punto di vista della progettazione, invece, copre undici frame logici. La differenza dipende dal test `authenticateShouldReturnSuccessfulResultForAcceptedDomainValues()`, che accorpa in un unico metodo i tre frame relativi ai valori di `domain`: "Master", "" e `null`. I restanti test coprono i casi critici di `authenticate(...)`, cioè `authentication = null`, credenziali nulle e `principal` nullo, e i cinque frame progettati per `getAuthorities(...)`: richiesta diretta, richiesta in delega, `delegation key` vuota, `username` vuoto e `username` nullo.

L'esecuzione Maven ha prodotto:

```
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
```

La build è terminata con esito positivo. I test attivi risultano quindi coerenti con gli oracoli implementati; dal punto di vista della progettazione, i 9 metodi JUnit coprono 11 frame logici, poiché un metodo accorpa i tre valori previsti per `domain`.

Questo risultato non dimostra l'assenza assoluta di difetti. In particolare, i casi `authentication = null` e credenziali nulle sono stati classificati come *characterization test* (AT4, AT5): l'oracolo black-box iniziale prevedeva un rifiuto controllato dell'input assente o incompleto, mentre l'implementazione corrente solleva una `NullPointerException`. Questo comportamento è stato quindi documentato come comportamento osservato, non come requisito desiderabile. Tali casi sono indicati come possibili fragilità nella gestione del token nullo e delle credenziali assenti. La criticità è particolarmente rilevante perché riguarda il confine di ingresso di un metodo di autenticazione. Da un punto di vista black-box, un token assente o privo di credenziali non dovrebbe portare a un'autenticazione positiva; ci si aspetterebbe piuttosto un fallimento esplicito, una risposta negativa oppure un'eccezione coerente con il dominio di sicurezza. Una `NullPointerException`, invece, rende meno chiara la semantica del fallimento: segnala l'accesso a un riferimento nullo più che una decisione esplicita del componente di rifiutare l'autenticazione.

Questa osservazione non basta, da sola, a classificare il comportamento come bug certo. La documentazione disponibile non specifica in modo esplicito quale eccezione debba essere sollevata per `authentication = null` o `credentials = null`; inoltre, si potrebbe sostenere che tali input violino una preconditione implicita del metodo e che il chiamante debba garantire la presenza di un token valido. Tuttavia, nel contesto di un componente di sicurezza, una gestione più robusta sarebbe preferibile: il metodo dovrebbe esplicitare il rifiuto dell'input assente o incompleto, evitando che il fallimento emerga come effetto collaterale non controllato.

Dopo questa esecuzione, JaCoCo è stato usato solo come misura di adequacy a posteriori. Il report mostra una copertura complessiva ancora parziale sulla classe: circa 24% di instruction coverage e 28% di branch coverage, come mostrato nella Figura 26.

2.4 Fase 2: Raffinamento strutturale guidato da JaCoCo

L'ispezione dei rami non coperti ha guidato una seconda iterazione coverage-aware, senza inseguire meccanicamente il 100% di coverage. Sono stati aggiunti test solo quando il ramo scoperto rappresentava un comportamento osservabile e coerente con lo scope già definito.

I raffinamenti sono stati raccolti nella classe `AuthDataAccessorWBTest`. Per `authenticate(...)` sono stati aggiunti tre casi: password rifiutata, utente sospeso e status non ammesso. Il primo completa il caso nominale di autenticazione positiva verificando risultato negativo, incremento dei failed logins e salvataggio dell'utente; gli altri due coprono forme di rifiuto controllato rilevanti nel dominio del metodo.

Per `getAuthorities(...)`, JaCoCo ha suggerito di raffinare la categoria `username` pieno distinguendo utenti ordinari e identità speciali del sistema. Sono stati quindi aggiunti due test sui valori `anonymous` e `admin`. Il primo verifica l'autorità `ANONYMOUS`; il secondo verifica che `admin` sia trattato come identità speciale, senza lookup ordinario tramite `userDAO`. L'oracolo su `admin` non impone una dimensione minima del set di `authorities`, per evitare dipendenze fragili dal popolamento statico/configurativo delle entitlements.

Dopo i raffinamenti, la suite manuale comprende 11 frame logici Black-Box e 5 test coverage-aware, tutti eseguiti con successo. La coverage complessiva sale a circa 28% di instruction coverage e 36% di branch coverage. L'aumento globale resta limitato perché molti metodi rimangono fuori scope; sui due metodi target, però, il miglioramento è più significativo: `authenticate(...)` passa circa dal 59% al 73%, mentre `getAuthorities(...)` passa circa dall'83% al 94%. Come riportato in Figure 27, 28, 29, 30 e 31.

I rami residui richiederebbero nuove partizioni su attributi di autenticazione diversi da `username`, `pass-through resources`, deleghe con ruoli propri, ruoli, realm e helper non selezionati nella fase iniziale. Coprire tali percorsi solo per aumentare JaCoCo avrebbe trasformato l'analisi in testing White-Box indiscriminato.

2.5 Fase 3: Adeguatezza semantica con PIT

Dopo il raffinamento guidato da JaCoCo, la suite di `AuthDataAccessor` è stata valutata con PIT per misurare la forza degli oracoli.

La suite sottoposta alla prima run PIT comprendeva i test Black-Box e i cinque raffinamenti White-Box.

La prima esecuzione ha generato 102 mutanti: 26 uccisi, 74 `NO_COVERAGE` e 2 `SURVIVED`. La *Mutation Coverage* risulta pari al 25%, mentre la *Test Strength* sui mutanti effettivamente coperti è pari al 93%. Il riepilogo è riportato nelle Figure 32, 33 e 34. I mutanti `NO_COVERAGE` indicano codice non attraversato, spesso relativo a metodi fuori scope come JWT, SSO, audit, rimozione di token scaduti o helper interni. I mutanti `SURVIVED`, invece, sono più rilevanti semanticamente: il codice mutato viene eseguito, ma gli oracoli non rilevano l'alterazione.

Il mutante sopravvissuto più significativo riguardava il controllo dello stato dell'account nel flusso di autenticazione. Per rafforzare l'oracolo è stato aggiunto un test dedicato al caso di utente con flag `isSuspended` nullo ma status ammesso (`authenticateShouldAcceptUserWithNullSuspendedFlagWhenStatusIsAllowed`). Il caso è coerente con lo scope perché riguarda direttamente la distinzione tra utente autenticabile e utente da rifiutare. Il dettaglio del mutante è riportato nella Figura 35.

Dopo il raffinamento mutation-driven, la suite finale arriva a 17 test verdi: 11 frame logici Black-Box, 5 raffinamenti coverage-aware e 1 raffinamento PIT. I mutanti sopravvissuti, tra quelli coperti dai test, scendono da 2 a 1; la run finale registra 27 mutanti uccisi su 28 coperti e una *Test Strength* pari al 96%, come mostrato nella Figura 36.

Il mutante `SURVIVED` residuo è localizzato in `getAdminAuthorities(...)` e riguarda la costruzione delle `authorities` amministrative. Non è stato trasformato in un ulteriore test perché richiederebbe un oracolo specifico sul contenuto delle entitlements amministrative, dipendente da `EntitlementsHolder` e dalla configurazione globale. Questo rappresenterebbe una nuova progettazione sulle `authorities` amministrative, non un semplice completamento della `category partition` corrente.

La stessa logica è stata applicata ai mutanti `NO_COVERAGE`. Inseguirli solo per aumentare il mutation score avrebbe allargato lo scope verso JWT, audit, token scaduti, `pass-through resources`, ruoli, realm e configurazioni esterne, trasformando PIT in testing White-Box indiscriminato.

2.6 Generazione automatica con Randoop

Randoop è stato eseguito sulla sola classe `org.apache.syncope.core.spring.security.AuthDataAccessor`, con time budget di 60 secondi. Il tool ha prodotto 1640 regression test, tutti eseguiti con successo, senza failure, errori o test saltati.

Il risultato quantitativo, però, è limitato rispetto allo scopo della fase. La suite Randoop-only raggiunge circa 11% di instruction coverage e circa 1% di branch coverage sulla classe, ma non esercita i due metodi target: `authenticate(...)` e `getAuthorities(...)` restano allo 0% di coverage, come mostrato nelle Figure 37, 38 e 39. Anche PIT conferma lo stesso limite: la mutation coverage resta molto bassa e i mutanti dei metodi target rimangono classificati come `NO_COVERAGE`, come riportato nelle Figure 40, 41 e 42.

Questo esito non indica un fallimento generale di Randoop, ma un limite rispetto al SUT considerato. `AuthDataAccessor` richiede fixture semanticamente coerenti con il dominio Syncope: `DAO`, `SecurityProperties`, `ConfParamOps`, deleghe, utenti, credenziali, status e authorities speciali. Randoop genera sequenze dalle API disponibili, ma non costruisce automaticamente mock configurati su tali scenari; di conseguenza tende a esercitare porzioni più lineari o periferiche della classe, senza raggiungere i flussi centrali di autenticazione e autorizzazione.

Per questo motivo, la suite Randoop è stata documentata come evidenza sperimentale, ma non promossa a suite principale.

2.7 Generazione tramite LLM

Continuando la generazione dei test automatici su `AuthDataAccessor`, è stato condotto anche un esperimento di generazione automatica tramite LLM.

Come per la prima classe, gli approcci zero-shot si sono rivelati insufficienti: fornendo solo il nome della classe, il modello generava test generici e poco contestualizzati; fornendo il codice sorgente, provava invece a cablare dipendenze complesse con errori di firme, tipi e API non presenti nel progetto.

È stata quindi adottata una strategia *Guided Tree-of-Thought*; il prompt principale è riportato nel Listato 3.

La suite LLM stabilizzata contiene 10 test JUnit eseguibili. I test non si limitano ai due metodi target, ma esplorano anche comportamenti laterali: provider JWT, rimozione di token scaduti, utenti speciali in `getAuthorities(...)`, flussi positivi e negativi di autenticazione, delega e audit. La suite ha richiesto un repair minimo: correzioni di compilazione e stabilizzazione di un oracolo troppo forte sul caso `admin`. In particolare, l'asserzione iniziale sul set non vuoto di authorities amministrative è stata sostituita con un controllo più robusto: set non nullo e nessun lookup ordinario tramite `userDAO` o `delegationDAO`. Il repair non modifica il codice di produzione né aggiunge nuovi casi manuali.

Valutata isolatamente, la suite LLM copre porzioni reali della classe, ma resta meno efficace della suite manuale sui due metodi principali: 22% di instruction coverage, 16% di branch coverage, 63 linee coperte su 219 e 14 metodi coperti su 46. La valutazione combinata è più significativa: eseguendo insieme suite manuale finale e LLM Guided Tree, la coverage sale al 40% di instruction coverage e al 47% di branch coverage, con 102 linee coperte su 219 e 18 metodi coperti su 46, come mostrato nelle Figure 43 e 44. L'incremento deriva soprattutto da metodi accessori come `getJWTSSOProvider(...)`, `removeExpired(...)`, `audit(...)` e percorsi ausiliari di `getAuthorities(...)`.

Anche PIT conferma questa lettura. Isolatamente, la suite LLM uccide 17 mutanti, lascia 5 sopravvivere e 80 `NO_COVERAGE`, con *Mutation Coverage* del 17% e *Test Strength* del 77%, come mostrato nella Figura 45. La suite manuale rimane più forte sulla porzione coperta, con 27 mutanti uccisi, 1 sopravvissuto e *Test Strength* del 96%. Tuttavia, la configurazione manuale + LLM uccide 38 mutanti, riduce i `NO_COVERAGE` a 59 e porta la *Mutation Coverage* complessiva al 37%, cioè 11 mutanti uccisi in più rispetto alla sola suite manuale, come riportato nella Figura 46.

La suite LLM non può quindi sostituire la suite manuale: non nasce da una partizione esplicita del dominio e non offre lo stesso controllo sugli oracoli dei comportamenti critici. Tuttavia, rispetto a Randoop, fornisce un contributo più significativo, perché costruisce fixture Mockito sufficientemente coerenti da raggiungere porzioni reali della classe e uccidere mutanti aggiuntivi. Per questo viene considerata un supporto incrementale, utile ad ampliare l'esplorazione del componente, ma non sufficiente da sola a sostituire una progettazione black-box e mutation-driven.

2.8 Generazione automatica con EvoSuite

In conclusione è stata valutata anche la generazione tramite EvoSuite. Come già osservato per `DefaultJobManager`, l'utilizzo diretto sul progetto corrente presenta problemi di compatibilità con bytecode e classpath. Per `AuthDataAccessor` è stato trovato e usato un artifact legacy ufficiale, `syncope-core-spring:2.1.0`, compilato con bytecode Java 8, cioè major version 52.

Questo artifact contiene `org.apache.syncope.core.spring.security.AuthDataAccessor` e ha permesso a EvoSuite 1.2.0 di completare la generazione, producendo una suite di 16 test con relativo scaffolding. Tuttavia, quando i file generati sono stati copiati nel modulo `core/spring` della versione corrente di Syncope, la compilazione si è arrestata in fase `testCompile`. Gli errori non riguardano asserzioni fallite, ma API non più presenti o profondamente cambiate, come `org.apache.cxf.rs.security.jose.jwt`, `AuditElements` e firme non compatibili con la versione target.

Il disallineamento emerge anche dal costruttore della classe. La versione corrente di `AuthDataAccessor` non espone un costruttore vuoto, ma richiede 15 collaboratori espliciti. L'errore `Expected 15 arguments but found 0`, riportato nella Figura 47, mostra quindi che la suite generata è coerente con una superficie API storica, ma non con il target sperimentale corrente.

La causa del fallimento è quindi un disallineamento tra la versione usata per la generazione e la versione usata per la validazione. EvoSuite ha generato test coerenti con la superficie API osservata in Syncope 2.1.0, ma tali test vengono poi compilati contro una classe profondamente evoluta. Nel passaggio alla versione corrente sono cambiate dipendenze, tipi di dominio e modalità di costruzione del SUT: ad esempio `AuditElements` è stato sostituito da `OpEvent`, alcune entità e DAO sono stati riorganizzati, la classe corrente espone record di risultato dedicati come `UsernamePasswordAuthResult` e `JWTAuthResult`, e il costruttore richiede ora 15 dipendenze esplicite.

Il problema non può essere risolto aggiungendo semplicemente dipendenze legacy: questo introdurrebbe API estranee alla versione attuale e renderebbe i test semanticamente legati alla vecchia implementazione. Un porting manuale sarebbe possibile, aggiornando import, tipi JWT, costruttore e asserzioni, ma a quel punto non si valterebbe più EvoSuite come generatore automatico, bensì una suite EvoSuite riscritta a mano.

Per questo motivo, EvoSuite viene riportato come N/A. La generazione su artifact legacy è avvenuta, ma la suite prodotta non è compilabile contro `AuthDataAccessor` nella versione corrente senza un porting manuale sostanziale; di conseguenza non viene inclusa nelle metriche JaCoCo o PIT.

2.9 Generazione delle varianti refactorizzate

Dopo la stabilizzazione della suite di test su `AuthDataAccessor`, è stata avviata la fase di refactoring automatico come per `DefaultJobManager`.

La progressione sperimentale segue lo schema già adottato: C_1 è stata generata senza test, C_2 con i test Black-Box, C_3 anche con i raffinantissimi Control-Flow guidati da JaCoCo e C_4 anche con il test derivato dal mutation testing. La suite LLM non è stata usata come contesto di generazione, perché rappresenta una suite automatica distinta e non una famiglia di test manuali BB, CF o MT; è stata invece mantenuta come validazione aggiuntiva dopo la generazione.

La Tabella 18 riporta il contesto di test fornito nel prompt per ciascuna variante.

2.10 Validazione delle varianti tramite suite di regressione

Dopo la generazione, ciascuna variante di `AuthDataAccessor` è stata copiata temporaneamente nel path originale della classe ed eseguita con le suite sviluppate su C_0 . La matrice degli output osservati è riportata nella Tabella 19.

Il risultato mostra che tutte le varianti, da C_1 a C_4 , superano tutte le suite eseguibili costruite su C_0 : Black-Box, Control-Flow, Mutation Testing, LLM e Randoop. Rispetto alla matrice Pass/Fail, quindi, non emergono regressioni osservabili. A differenza di quanto accaduto per `DefaultJobManager`, in questo caso il refactoring non ha introdotto una modifica intercettata dagli oracoli della suite.

Il passaggio delle suite non implica equivalenza assoluta rispetto a C_0 , ma indica che, nei casi coperti, non emergono regressioni osservabili. Le varianti preservano quindi i comportamenti verificati dalla regressione disponibile: autenticazione, autorizzazione, raffinamenti strutturali, caso mutation-driven e scenari aggiuntivi introdotti dalle suite automatiche.

2.11 Confronto dei test automatici generati sulle varianti

Dopo la validazione tramite suite di regressione, è stata compilata la terza matrice sperimentale, dedicata ai test automatici generati ex novo su C_0 e sulle varianti C_1 – C_4 .

Le suite considerate sono Randoop (RND) e LLM Guided Tree-of-Thought (LLM); EvoSuite è indicato come N/A, per i motivi accennati in precedenza. Le metriche usate sono la *Line Coverage* di JaCoCo e il *Mutation Score* di PIT. Per ogni variante è stato calcolato il delta rispetto alla baseline della stessa famiglia di test su C_0 :

La matrice completa è riportata nella Tabella 20. Il primo risultato riguarda Randoop: a differenza di quanto osservato su `DefaultJobManager`, ottiene coverage non nulla su `AuthDataAccessor`, ma resta stabile e basso su tutte le varianti, intorno al 14–15% di line coverage e al 4% di mutation score. Questo indica che il refactoring non rende la classe significativamente più esplorabile da Randoop.

I risultati LLM sono più variabili. Il dettaglio delle metriche JaCoCo e PIT per le suite LLM generate sulle singole varianti è riportato nella Tabella 21. La baseline su C_0 è stata ottenuta usando una suite LLM rigenerata per includere anche il comportamento di `authenticate(...)`, che nella prima generazione non risultava coperto in modo sufficiente. In questa configurazione, C_0 ottiene il 41.55% di line coverage e il 19% di mutation score. C_1 risulta più debole, con $\Delta Coverage = -13.87\%$ e $\Delta Mutation = -1\%$. Le varianti più interessanti sono C_2 e C_3 : entrambe hanno coverage inferiore a C_0 , ma mutation score superiore, rispettivamente con $\Delta Mutation = +4\%$ e $\Delta Mutation = +2\%$. Questo mostra che maggiore coverage non coincide sempre con maggiore efficacia semantica: le suite possono attraversare meno linee, ma intercettare mutanti più significativi. C_4 è invece la variante meno favorevole, con $\Delta Coverage = -14.56\%$ e $\Delta Mutation = -2\%$.

Il dettaglio conferma inoltre che il valore di *Test Strength* non segue necessariamente l'andamento della mutation coverage complessiva: ad esempio C_1 mostra un mutation score leggermente inferiore a C_0 , ma una test strength più alta, perché i mutanti effettivamente coperti dalla suite vengono uccisi con maggiore efficacia. Questo rafforza la distinzione tra ampiezza della copertura e forza degli oracoli sui punti realmente esercitati.

La lettura della Tabella 20 deve rimanere distinta dalla matrice Pass/Fail. La Tabella 19 indica che le varianti non introducono regressioni osservabili rispetto agli oracoli disponibili; la terza matrice mostra invece che il refactoring non produce un miglioramento uniforme della generazione automatica. Randoop resta quasi invariato, mentre LLM migliora selettivamente solo su alcune varianti, in particolare C_2 e C_3 .

Nel complesso, per `AuthDataAccessor` le varianti risultano stabili rispetto alla regressione disponibile, ma la qualità dei test automatici generati resta dipendente dal tool e dalla capacità di costruire scenari coerenti con il dominio.

Anche per `AuthDataAccessor` è stata osservata la qualità statica delle suite LLM generate, applicando lo stesso criterio già descritto per `DefaultJobManager`, ovvero l'analisi dei code smells. Sono state quindi escluse le segnalazioni dovute al mismatch tra path e package Java, mentre sono state considerate le issue effettive del codice generato: naming non conforme, duplicazione di literal, variabili inutilizzate, assegnazioni inutili, uso non idiomatico delle asserzioni e codice commentato. Il riepilogo è riportato nella Tabella 22; gli screenshot sono riportati nelle Figure 48–52.

I risultati confermano che manutenibilità ed efficacia non coincidono necessariamente. C_1 presenta poche issue considerate, ma ottiene risultati deboli in coverage e mutation score; al contrario, C_3 presenta più issue statiche, ma è una delle suite LLM più interessanti per adequacy, perché mantiene una coverage vicina alla baseline e migliora il mutation score rispetto a C_0 . Gli smell sono quindi un indicatore del costo di manutenzione della suite generata, non un criterio sufficiente per giudicare da soli la qualità della generazione.

3 Allegati, Tabelle e Figure

La presente sezione raccoglie i dati crudi e le matrici strutturali a supporto delle analisi analitiche espresse nei capitoli precedenti del report.

Table 1: Rappresentanti concreti per il metodo `execute`

Parametro	Classe	Rappresentante
task	Nulla	null
task	Valido attivo	Mock SchedTask: isActive()=true, job delegate valido
task	Valido inattivo	Mock SchedTask: isActive()=false
task	Senza job delegate	Mock SchedTask: isActive()=true, job delegate assente
startAt	Nulla	null
startAt	Passato	OffsetDateTime.now().minusDays(1)
startAt	Presente	OffsetDateTime.now()
startAt	Futuro	OffsetDateTime.now().plusDays(1)
executor	Nulla	null
executor	Stringa vuota	""
executor	Stringa non vuota	"admin"
dryRun	Simulazione	true
dryRun	Esecuzione normale	false
jobData	Nulla	null
jobData	Mappa vuota	Map.of()
jobData	Mappa non vuota	Map.of("sampleKey", "sampleValue")

Table 2: Matrice dei test frame manuali per il metodo `execute`

ID	Variazione	Stato	Oracolo
TF1	Task attivo, startAt futuro, executor valorizzato, jobData vuota	Valido	O1
TF2	startAt presente; dryRun=true; jobData non vuoto	Valido	O2
TF3	Task inattivo; startAt=null	Valido	O3
TF4	Task nullo	Invalido	O4
TF5	Task senza job delegate	Invalido	O4
TF6	startAt nel passato	Invalido	O5
TF7	executor=null	Invalido	O6
TF8	executor=""	Invalido	O6
TF9	jobData=null	Invalido	O7

Table 3: Legenda degli oracoli per il metodo `execute`

Codice	Comportamento atteso
O1	Schedulazione del job; nessuna eccezione
O2	Schedulazione del job; dati di contesto propagati; nessuna eccezione
O3	Nessuna schedulazione; nessuna creazione del job; nessuna eccezione
O4	Eccezione; nessuna schedulazione
O5	Oracolo iniziale: schedulazione non valida o rifiuto dell'input
O6	Oracolo iniziale: rifiuto dell'input
O7	Oracolo iniziale: rifiuto controllato o gestione protetta dell'input

Table 4: Rappresentanti concreti per il metodo `load`

Parametro	Classe	Rappresentante
domain	Stringa piena	"test-domain"
domain	Nulla	null
domain	Stringa vuota	" "

Table 5: Matrice dei test frame manuali per il metodo `load`

ID	Variazione	Stato	Oracolo
D1	Caso nominale; domain pieno	Valido	L-O1
D2	domain=null	Limite	L-O2
D3	domain=""	Limite	L-O2

Table 6: Legenda degli oracoli per il metodo `load`

Codice	Comportamento atteso
L-O1	Nessuna eccezione inattesa
L-O2	Oracolo iniziale: rifiuto atteso dell'input

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
MacroJobDelegate		0%		0%	64 64	146 146	27 27	1 1
DefaultJobManager		36%		42%	32 43	108 169	15 23	0 1
GroupMemberProvisionTaskJobDelegate		0%		0%	25 25	81 81	5 5	1 1
SyncopeTaskScheduler		1%		0%	35 36	56 57	25 26	0 1
AbstractSchedTaskJobDelegate		0%		0%	17 17	58 58	14 14	1 1
TaskJob		3%		0%	9 10	29 30	5 6	0 1
Job		3%		0%	8 9	23 24	7 8	0 1

Figure 1: Overview della coverage strutturale sulla classe `DefaultJobManager` dopo le suite Black-Box.

Element	Missed Instructions	Cov.	Missed Branches	Cov.
• lambda\$load\$0(String)		38%		25%
• load(String)		11%		25%
• lambda\$unload\$0()		0%		n/a
• register(String, Report, String, OffsetDateTime, String, boolean)		0%		0%
• lambda\$unload\$1(SchedTask)		0%		n/a
• register(String, SchedTask, String, OffsetDateTime, String, boolean, Map)		79%		80%
• lambda\$load\$1()		0%		0%
• registerJob(JobExecutionContext, Class, String, OffsetDateTime)		60%		50%
• lambda\$unload\$2(Report)		0%		n/a
• register(SchedTask, String)		0%		n/a
• unregisterJob(String)		0%		n/a
• register(Report, String)		0%		n/a
• execute(Report, OffsetDateTime, String, boolean)		0%		n/a
• lambda\$register\$1(Implementation)		0%		n/a
• lambda\$register\$0(Implementation)		0%		n/a
• unregister(Task)		0%		n/a
• unregister(Report)		0%		n/a
• unload(String)		0%		n/a
• isRunning(String)		91%		50%
• getOrder()		0%		n/a
• DefaultJobManager.DomainHolder.SyncopTaskScheduler.JobStatusDAO.TaskDAO.ReportDAO.ImplementationDAO.TaskUtilsFactory.ConfParamOps.SecurityProperties.ConfigurableApplicationContext)		100%		n/a
• execute(SchedTask, OffsetDateTime, String, boolean, Map)		100%		n/a
• static (...)		100%		n/a
Total	444 of 700	36%	23 of 40	42%

Figure 2: Dettaglio della coverage sui singoli metodi di DefaultJobManager.

```

194.         final String executor,
195.         final boolean dryRun,
196.         final Map<String, Object> jobData) {
197.
198.     register(
199.         AuthContextUtils.getDomain(),
200.         task,
201.         null,
202.         startAt,
203.         executor,
204.         dryRun,
205.         jobData);
206.
207.
208.     @Override
209.     public void register(
210.         final SchedTask task,
211.         final String executor) {
212.
213.         register(
214.             AuthContextUtils.getDomain(),
215.             task,
216.             task.getCronExpression(),
217.             null,
218.             executor,
219.             false,
220.             Map.of());
221.     }
222.
223.     protected void register(
224.         final String domain,
225.         final Report report,
226.         final String cronExpression,
227.         final OffsetDateTime startAt,
228.         final String executor,
229.         final boolean dryRun) {
230.
231.         if (!report.isActive()) {
232.             LOG.debug("Report {} {} not active, skipping job registration", report.getKey(), report.getName());
233.             return;
234.         }
235.
236.         JobExecutionContext context = new JobExecutionContext(
237.             domain,
238.             JobName.getJobName(report),
239.             executor,
240.             dryRun);
241.         context.getData().put(JobManager.REPORT_KEY, report.getKey());
242.         context.getData().put(JobManager.DELEGATE_IMPLEMENTATION, report.getJobDelegate().getKey());
243.
244.         registerJob(
245.             context,
246.             ReportJob.class,
247.             cronExpression,
248.             startAt);
249.     }
250.

```

Figure 3: Copertura strutturale sul metodo register(...) dentro execute(...) dopo i test Black-Box.

```

300. public void load(final String domain) {
301.     AuthContextUtils.runAsAdmin(domain, () -> {
302.         // 1. jobs for SchedTasks
303.         Set<SchedTask> tasks = new HashSet<>(taskDAO.findAll(TaskType.SCHEDULED));
304.         tasks.addAll(taskDAO.findAll(TaskType.PULL));
305.         tasks.addAll(taskDAO.findAll(TaskType.PUSH));
306.         tasks.addAll(taskDAO.findAll(TaskType.MACRO));
307.         tasks.addAll(taskDAO.findAll(TaskType.LIVE_SYNC));
308.
309.         boolean loadException = false;
310.         for (Iterator<SchedTask> it = tasks.iterator(); it.hasNext() && !loadException;) {
311.             SchedTask task = it.next();
312.
313.             LOG.debug("Loading job for {} Task {}", task.getKey(), task.getName());
314.             taskUtilsFactory.getInstance(task).getType(), task.getKey(), task.getName();
315.
316.             try {
317.                 register(
318.                     domain,
319.                     task,
320.                     task.getCronExpression(),
321.                     null,
322.                     securityProperties.getAdminUser(),
323.                     false,
324.                     Map.of());
325.             } catch (Exception e) {
326.                 LOG.error("While loading job instance for task {}", task.getKey(), e);
327.                 loadException = true;
328.             }
329.         }
330.
331.         // 2. jobs for Reports
332.         loadException = false;
333.         for (Iterator<? extends Report> it = reportDAO.findAll().iterator(); it.hasNext() && !loadException;) {
334.             Report report = it.next();
335.
336.             LOG.debug("Loading job for Report {}", report.getKey(), report.getName());
337.
338.             try {
339.                 register(
340.                     domain,
341.                     report,
342.                     report.getCronExpression(),
343.                     null,
344.                     securityProperties.getAdminUser(),
345.                     false);
346.             } catch (Exception e) {
347.                 LOG.error("While loading job instance for report {}", report.getName(), e);
348.                 loadException = true;
349.             }
350.         }
351.     });
352.
353.     if (SyncopeConstants.MASTER_DOMAIN.equals(domain)) {
354.         String notificationJobCronExp = AuthContextUtils.callAsAdmin(SyncopeConstants.MASTER_DOMAIN, () -> {
355.             String result = StringUtils.EMPTY;

```

Figure 4: Copertura del primo blocco iterativo del metodo load(...), relativo ai task.

```

    if (SyncopConstants.MASTER_DOMAIN.equals(domain)) {
        String notificationJobCronExp = AuthContextUtils.callAsAdmin(SyncopConstants.MASTER_DOMAIN, () -> {
            String result = StringUtils.EMPTY;

            String conf = confParamOps.get(
                SyncopConstants.MASTER_DOMAIN, "notificationjob.cronExpression", null, String.class);
            if (conf == null) {
                result = NotificationJob.DEFAULT_CRON_EXP;
            } else if (!StringUtils.EMPTY.equals(conf)) {
                result = conf;
            }
            return result;
        });

        // 3. NotificationJob
        if (StringUtils.isBlank(notificationJobCronExp)) {
            LOG.debug("Empty value provided for {}'s cron, not scheduling", NotificationJob.class.getSimpleName());
        } else {
            LOG.debug("{}'s cron expression: {} - scheduling",
                NotificationJob.class.getSimpleName(), notificationJobCronExp);

            JobExecutionContext context = new JobExecutionContext(
                domain,
                NOTIFICATION_JOB,
                securityProperties.getAdminUser(),
                false);

            try {
                registerJob(
                    context,
                    NotificationJob.class,
                    notificationJobCronExp,
                    null);
            } catch (Exception e) {
                LOG.error("While loading {} instance", NotificationJob.class.getSimpleName(), e);
            }
        }

        // 4. SystemLoadReporterJob (fixed schedule, every minute)
        LOG.debug("Registering {}", SystemLoadReporterJob.class);

        JobExecutionContext context = new JobExecutionContext(
            domain,
            StringUtils.uncapitalize(SystemLoadReporterJob.class.getSimpleName()),
            securityProperties.getAdminUser(),
            false);

        try {
            registerJob(
                context,
                SystemLoadReporterJob.class,
                "0 * * * * ?",
                null);
        } catch (Exception e) {
            LOG.error("While loading {} instance", SystemLoadReporterJob.class.getSimpleName(), e);
        }
    }
}

```

Figure 5: Copertura del ramo relativo al dominio master nel metodo load(...).

org.apache.syncope.core.provisioning.java.job

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctxty	Missed Lines	Missed Methods	Missed Classes
MacroJobDelegate		0%		0%	64 64	146 146	27 27	1 1
GroupMemberProvisionTaskJobDelegate		0%		0%	25 25	81 81	5 5	1 1
SyncopTaskScheduler		1%		0%	35 36	56 57	25 26	0 1
DefaultJobManager		64%		77%	22 43	62 169	13 23	0 1
AbstractSchedTaskJobDelegate		0%		0%	17 17	58 58	14 14	1 1
TaskJob		3%		0%	9 10	29 30	5 6	0 1
Job		3%		0%	8 9	23 24	7 8	0 1
AfterHandlingJob		0%		0%	7 7	28 28	6 6	1 1
JobStatusUpdater		0%		0%	7 7	18 18	5 5	1 1
SystemLoadReporterJob		7%		n/a	3 4	11 12	3 4	0 1
LiveSyncJob		0%		0%	5 5	10 10	4 4	1 1
ExpiredAccessTokenCleanup		0%		0%	3 3	5 5	2 2	1 1
ExpiredBatchCleanup		0%		0%	3 3	5 5	2 2	1 1
TaskJob.Status		0%		n/a	1 1	3 3	1 1	1 1
SyncopTaskSchedulerValue		0%		n/a	1 1	1 1	1 1	1 1
SyncopTaskSchedulerKey		0%		n/a	1 1	1 1	1 1	1 1
Total	2.610 of 3.080	15%	164 of 195	15%	211 236	537 648	121 135	11 16

Figure 6: Overview della coverage strutturale sulla classe DefaultJobManager dopo le suite WhiteBox.


```

351.     });
352.
353.     if (SyncopeConstants.MASTER_DOMAIN.equals(domain)) {
354.         String notificationJobCronExp = AuthContextUtils.callAsAdmin(SyncopeConstants.MASTER_DOMAIN, () -> {
355.             String result = StringUtils.EMPTY;
356.
357.             String conf = confParamOps.get(
358.                 SyncopeConstants.MASTER_DOMAIN, "notificationjob.cronExpression", null, String.class);
359.             if (conf == null) {
360.                 result = NotificationJob.DEFAULT_CRON_EXP;
361.             } else if (!StringUtils.EMPTY.equals(conf)) {
362.                 result = conf;
363.             }
364.             return result;
365.         });
366.
367.         // 3. NotificationJob
368.         if (StringUtils.isBlank(notificationJobCronExp)) {
369.             LOG.debug("Empty value provided for {}'s cron, not scheduling", NotificationJob.class.getSimpleName());
370.         } else {
371.             LOG.debug("{}'s cron expression: {} - scheduling",
372.                 NotificationJob.class.getSimpleName(), notificationJobCronExp);
373.
374.             JobExecutionContext context = new JobExecutionContext(
375.                 domain,
376.                 NOTIFICATION_JOB,
377.                 securityProperties.getAdminUser(),
378.                 false);
379.             try {
380.                 registerJob(
381.                     context,
382.                     NotificationJob.class,
383.                     notificationJobCronExp,
384.                     null);
385.             } catch (Exception e) {
386.                 LOG.error("While loading {} instance", NotificationJob.class.getSimpleName(), e);
387.             }
388.         }
389.
390.         // 4. SystemLoadReporterJob (fixed schedule, every minute)
391.         LOG.debug("Registering {}", SystemLoadReporterJob.class);
392.
393.         JobExecutionContext context = new JobExecutionContext(

```

Figure 7: Copertura del primo blocco iterativo del metodo load(...), relativo ai task.

DefaultJobManager

Element	Missed Instructions	Cov.	Missed Branches	Cov.
• lambda\$unload\$0()		0%		n/a
• lambda\$unload\$1(SchedTask)		0%		n/a
• register(String, SchedTask, String, OffsetDateTime, String, boolean, Map)		79%		80%
• lambda\$unload\$2(Report)		0%		n/a
• lambda\$load\$0(String)		87%		75%
• load(String)		83%		100%
• register(SchedTask, String)		0%		n/a
• unregisterJob(String)		0%		n/a
• register(Report, String)		0%		n/a
• execute(Report, OffsetDateTime, String, boolean)		0%		n/a
• register(String, Report, String, OffsetDateTime, String, boolean)		80%		50%
• registerJob(JobExecutionContext, Class, String, OffsetDateTime)		87%		87%
• lambda\$register\$1(Implementation)		0%		n/a
• lambda\$register\$0(Implementation)		0%		n/a
• unregister(Task)		0%		n/a
• unregister(Report)		0%		n/a
• unload(String)		0%		n/a
• lambda\$load\$1()		87%		75%
• isRunning(String)		91%		50%
• getOrder()		0%		n/a
• DefaultJobManager(DomainHolder, SyncopeTaskScheduler, JobStatusDAO, TaskDAO, ReportDAO, ImplementationDAO, TaskUtilsFactory, ConfParamOps, SecurityProperties, ConfigurableApplicationContext)		100%		n/a
• execute(SchedTask, OffsetDateTime, String, boolean, Map)		100%		n/a
• static (...)		100%		n/a
Total	246 of 700	64%	9 of 40	77%

Figure 8: Dettaglio della coverage sui singoli metodi di DefaultJobManager.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	63% 106/169	59% 32/54	86% 32/37

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.syncope.core.provisioning.java.job 1	1	63% 106/169	59% 32/54	86% 32/37

Figure 9: Riepilogo globale del report PIT prima del raffinamento, con evidenza di line coverage, mutation coverage e test strength.

111	1. negated conditional → SURVIVED Covering tests
112	1. removed call to org/apache/syncope/core/persistence/api/dao/JobStatusDAO::unlock → SURVIVED Covering tests
113	1. negated conditional → KILLED
114	2. replaced boolean return with false for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::isRunning → SURVIVED Covering tests
115	3. replaced boolean return with true for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::isRunning → KILLED
124	1. negated conditional → KILLED
131	1. removed call to org/apache/syncope/core/provisioning/java/job/Job::setContext → KILLED
134	1. negated conditional → KILLED
135	2. negated conditional → KILLED
137	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::register → KILLED
138	1. negated conditional → KILLED
140	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::schedule → KILLED
146	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::schedule → KILLED
156	1. negated conditional → KILLED
161	1. negated conditional → KILLED
162	1. negated conditional → SURVIVED Covering tests
164	1. replaced boolean return with false for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$register\$0 → NO_COVERAGE
165	2. replaced boolean return with true for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$register\$0 → NO_COVERAGE
166	1. negated conditional → SURVIVED Covering tests
168	1. replaced boolean return with false for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$register\$1 → NO_COVERAGE
169	2. replaced boolean return with true for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$register\$1 → NO_COVERAGE
172	1. negated conditional → KILLED
185	1. removed call to java/util/Map::putAll → KILLED
187	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::registerJob → KILLED
202	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → KILLED
217	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → NO_COVERAGE
235	1. negated conditional → KILLED
248	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::registerJob → KILLED
262	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → NO_COVERAGE
276	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → NO_COVERAGE
280	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::stop → NO_COVERAGE
281	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::delete → NO_COVERAGE
286	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::unregisterJob → NO_COVERAGE
291	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::unregisterJob → NO_COVERAGE
296	1. replaced int return with 0 for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::getOrder → NO_COVERAGE
301	1. removed call to org/apache/syncope/core/spring/security/AuthContextUtils::runAsAdmin → KILLED
310	1. negated conditional → KILLED
311	2. negated conditional → KILLED
317	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → KILLED
333	1. negated conditional → KILLED
334	2. negated conditional → KILLED
339	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → KILLED
353	1. negated conditional → KILLED
359	1. negated conditional → KILLED
361	1. negated conditional → KILLED
364	1. replaced return value with "" for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$load\$1 → KILLED
368	1. negated conditional → KILLED
380	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::registerJob → KILLED

Figure 10: Ripartizione dei mutanti per tipologia.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	63% 107/169	65% 35/54	92% 35/38

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.syncope.core.provisioning.java.job 1	1	63% 107/169	65% 35/54	92% 35/38

Figure 11: Riepilogo globale del report PIT dopo il raffinamento dei test.

org.apache.syncope.core.provisioning.java.job

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
MacroJobDelegate		0%		0%	64	146	27	1
DefaultJobManager		0%		0%	43	168	23	1
GroupMemberProvisionTaskJobDelegate		0%		0%	25	81	5	1
SyncopeTaskScheduler		0%		0%	36	57	26	1

Figure 12: Overview della coverage strutturale sulla classe DefaultJobManager con la suite Randoop.

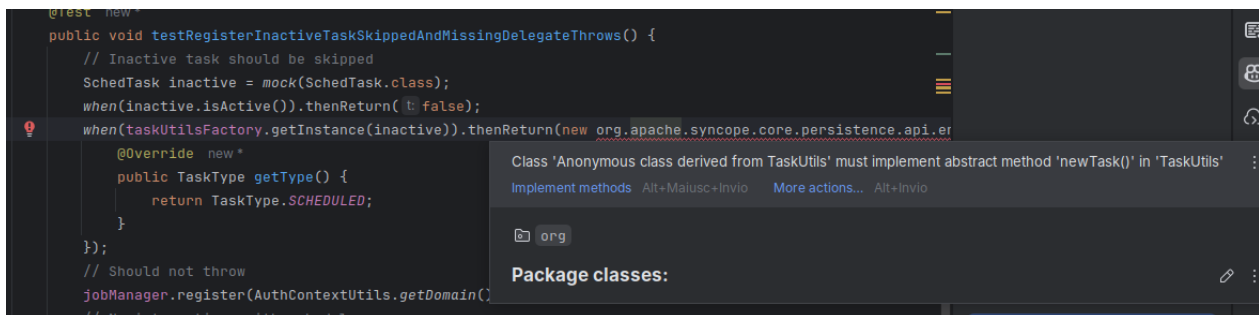


Figure 13: Evidenza degli errori di compilazione riscontrati durante l'esperimento Zero-Shot con Source Code.

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
MacroJobDelegate		0%		0%	64	146	27	1
DefaultJobManager		33%		50%	24	109	13	0
GroupMemberProvisionTaskJobDelegate		0%		0%	25	81	5	1
SyncopeTaskScheduler		10%		0%	36	57	26	1

Figure 14: JaCoCo: Copertura strutturale globale generata esclusivamente dalla suite LLM Guided Tree.

Element	Missed Instructions	Cov.	Missed Branches	Cov.
lambda\$load\$0(String)		0%		0%
load(String)		0%		0%
lambda\$unload\$0()		0%		n/a
register(String, Report, String, OffsetDateTime, String, boolean)		0%		0%
lambda\$unload\$1(SchedTask)		0%		n/a
register(String, SchedTask, String, OffsetDateTime, String, boolean, Map)		79%		80%
lambda\$load\$1()		0%		0%
lambda\$unload\$2(Report)		0%		n/a
register(SchedTask, String)		0%		n/a
register(Report, String)		0%		n/a
execute(Report, OffsetDateTime, String, boolean)		0%		n/a
lambda\$register\$1(Implementation)		0%		n/a
lambda\$register\$0(Implementation)		0%		n/a
unload(String)		0%		n/a
registerJob(JobExecutionContext, Class, String, OffsetDateTime)		100%		100%
DefaultJobManager(DomainHolder, SyncopeTaskScheduler, JobStatusDAO, TaskDAO, ReportDAO, ImplementationDAO, TaskUtilsFactory, ConfParamOps, SecurityProperties, ConfigurableApplicationContext)		100%		n/a
isRunning(String)		100%		100%
unregisterJob(String)		100%		n/a
execute(SchedTask, OffsetDateTime, String, boolean, Map)		100%		n/a
unregister(Task)		100%		n/a
unregister(Report)		100%		n/a
static {...}		100%		n/a
getOrder()		100%		n/a
Total	465 of 700	33%	20 of 40	50%

Figure 15: JaCoCo: Dettaglio della copertura sui singoli metodi ottenuta dalla suite LLM.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	35% 60/170	43% 23/54	88% 23/26

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.syncope.core.provisioning.java.job 1		35% 60/170	43% 23/54	88% 23/26

- [Project uses Spring, but the Arcmutate Spring plugin is not present.](#)

Report generated by [PIT 1.24.0 support](#)

Enhanced functionality available at [arcmutate.com](#)

Figure 16: PIT: Risultati della mutation analysis isolata sulla suite LLM, con il 43% di Mutation Coverage.

111	1. negated conditional → KILLED
112	1. removed call to org/apache/syncope/core/persistence/api/dao/JobStatusDAO::unlock → KILLED
	1. negated conditional → KILLED
114	2. replaced boolean return with false for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::isRunning → KILLED
	3. replaced boolean return with true for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::isRunning → KILLED
124	1. negated conditional → KILLED
131	1. removed call to org/apache/syncope/core/provisioning/java/job/Job::setContext → KILLED
134	1. negated conditional → KILLED
	2. negated conditional → KILLED
135	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::register → KILLED
137	1. negated conditional → KILLED
138	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::schedule → KILLED
140	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::schedule → KILLED
156	1. negated conditional → KILLED
161	1. negated conditional → KILLED
162	1. negated conditional → SURVIVED Covering tests
164	1. replaced boolean return with false for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$register\$0 → NO_COVERAGE
	2. replaced boolean return with true for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$register\$0 → NO_COVERAGE
166	1. negated conditional → SURVIVED Covering tests
168	1. replaced boolean return with false for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$register\$1 → NO_COVERAGE
	2. replaced boolean return with true for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$register\$1 → NO_COVERAGE
172	1. negated conditional → KILLED
185	1. removed call to java/util/Map::putAll → SURVIVED Covering tests
187	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::registerJob → KILLED
202	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → KILLED
217	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → NO_COVERAGE
235	1. negated conditional → NO_COVERAGE
248	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::registerJob → NO_COVERAGE
262	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → NO_COVERAGE
276	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → NO_COVERAGE
280	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::stop → KILLED
281	1. removed call to org/apache/syncope/core/provisioning/java/job/SyncopeTaskScheduler::delete → KILLED
286	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::unregisterJob → KILLED
291	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::unregisterJob → KILLED
296	1. replaced int return with 0 for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::getOrder → KILLED
301	1. removed call to org/apache/syncope/core/spring/security/AuthContextUtils::runAsAdmin → NO_COVERAGE
310	1. negated conditional → NO_COVERAGE
	2. negated conditional → NO_COVERAGE
317	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → NO_COVERAGE
333	1. negated conditional → NO_COVERAGE
	2. negated conditional → NO_COVERAGE
339	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::register → NO_COVERAGE
353	1. negated conditional → NO_COVERAGE
359	1. negated conditional → NO_COVERAGE
361	1. negated conditional → NO_COVERAGE
364	1. replaced return value with ** for org/apache/syncope/core/provisioning/java/job/DefaultJobManager::lambda\$load\$1 → NO_COVERAGE
368	1. negated conditional → NO_COVERAGE
380	1. removed call to org/apache/syncope/core/provisioning/java/job/DefaultJobManager::registerJob → NO_COVERAGE
382	

Figure 17: PIT: Dettaglio dei mutanti uccisi (verde) e sopravvissuti (rosso) a fronte dei test generati dall’LLM.

org.apache.syncope.core.provisioning.java.job

Element	Missed Instructions	Cov.	Missed Branches	Cov.
MacroJobDelegate		0%		0%
GroupMemberProvisionTaskJobDelegate		0%		0%
SyncopeTaskScheduler		1%		0%
AbstractSchedTaskJobDelegate		0%		0%
DefaultJobManager		69%		85%
TaskJob		3%		0%
Job		3%		0%
AfterHandlingJob		0%		0%

Figure 18: JaCoCo: overview della coverage su DefaultJobManager con suite manuale e suite LLM eseguite congiuntamente.

DefaultJobManager

Element	Missed Instructions	Cov.	Missed Branches	Cov.
lambda\$unload\$0()		0%		n/a
lambda\$unload\$1(SchedTask)		0%		n/a
register(String, SchedTask, String, OffsetDateTime, String, boolean, Map)		79%		80%
lambda\$unload\$2(Report)		0%		n/a
lambda\$load\$0(String)		87%		75%
load(String)		83%		100%
register(SchedTask, String)		0%		n/a
register(Report, String)		0%		n/a
execute(Report, OffsetDateTime, String, boolean)		0%		n/a
register(String, Report, String, OffsetDateTime, String, boolean)		80%		50%
lambda\$register\$1(Implementation)		0%		n/a
lambda\$register\$0(Implementation)		0%		n/a
unload(String)		0%		n/a
lambda\$load\$1()		87%		75%
registerJob(JobExecutionContext, Class, String, OffsetDateTime)		100%		100%
DefaultJobManager(DomainHolder, SyncopeTaskScheduler, JobStatusDAO, TaskDAO, ReportDAO, ImplementationDAO, TaskUtilsFactory, ConfParamOps, SecurityProperties)		100%		n/a
isRunning(String)		100%		100%
unregisterJob(String)		100%		n/a
execute(SchedTask, OffsetDateTime, String, boolean, Map)		100%		n/a

Figure 19: JaCoCo: dettaglio per metodo nella configurazione combinata manuale + LLM su DefaultJobManager.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	69% 116/169	74% 40/54	95% 40/42

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.syncope.core.provisioning.java.job	1	69% 116/169	74% 40/54	95% 40/42

- Project uses Spring, but the Arcmutate Spring plugin is not present.
- [JUnit 5 is on the classpath but the pitest junit 5 plugin is not installed.](#)

Report generated by [PIT 1.25.4 support](#)

Enhanced functionality available at [arcmutate.com](#)

Figure 20: PIT: risultati della configurazione combinata manuale + LLM su DefaultJobManager.

```

1 Imagine three different experts in software testing who are tasked with developing
2 comprehensive JUnit 4 test cases for the following Java class named DefaultJobManager.
3 They must follow these steps:
4 1. Extract and list all public and protected methods of the class, including their signatures.
5 2. For each method, identify its expected functionality by analyzing the source code.
6 3. Given the source code of the class and the listed methods, identify normal cases, edge
7 cases, and exception-handling scenarios that should be tested.
8 4. For each method, generate JUnit 4 test cases that check the method's functionality.
9 5. Merge all the individual test cases into a complete JUnit 4 test file named
10 DefaultJobManagerLLMGuidedTreeTest.java.
11 6. Add a few comments for the generated tests.
12
13 The generated test file must satisfy the following requirements:
14 - The package must be org.apache.syncope.core.provisioning.java.job.
15 - The test class name must be DefaultJobManagerLLMGuidedTreeTest.
16 - Use JUnit 4.
17 - Use Mockito for dependencies required by the constructor.
18 - Use org.mockito.junit.MockitoJUnitRunner.
19 - Do not use org.mockito.runners.MockitoJUnitRunner.
20 - Do not use org.mockito.Matchers.
21 - Do not use JUnit 5 annotations or assertions.
22 - Include a short comment for each generated test explaining the expected behavior.
23 - Prefer observable behavior and Mockito interaction checks.
24 - Avoid tests that only verify constants, Java reflection properties, Object methods, or
25 constructor existence.
26 - The output file must start with ###Test START### and finish with ###Test END###.
27
28 Important compilation guidance:
29 - DefaultJobManager has constructor dependencies and must be instantiated with mocked
30 collaborators.
31 - If a dependency type has many abstract methods, prefer Mockito mocks instead of anonymous
32 class implementations.
33 - Do not use TaskUtils as a lambda or anonymous functional interface. Mock TaskUtils and stub
34 getType().
35 - If beanFactory.createBean(TaskJob.class) is used, return a TaskJob mock, not a generic Job
36 mock.
37 - If beanFactory.createBean(ReportJob.class) is used, return a ReportJob mock, not a generic
38 Job mock.
39 - If a method is overloaded, use typed Mockito matchers to avoid ambiguous invocations.
40
41 The Java class is: DefaultJobManager and the source code is in "Testo incollato.txt".
42 ###Test START###

```

Listing 1: Prompt Guided Tree-of-Thought utilizzato per la generazione LLM dei test su DefaultJobManager

Table 7: Test sviluppati a partire da C_0 e input forniti all'LLM per la generazione delle varianti di DefaultJobManager

Classe	Versione	Smell	Test BB	Test CF	Test MT	Test RND	Test ES	Test LLM
C_0	Originale	baseline	Yes	Yes	Yes	Yes	N/A	Yes
C_1	Refactored	Yes	No	No	No	No	N/A	No
C_2	Refactored	Yes	Yes	No	No	No	N/A	No
C_3	Refactored	Yes	Yes	Yes	No	No	N/A	No
C_4	Refactored	Yes	Yes	Yes	Yes	No	N/A	No

Table 8: Output osservati eseguendo i test sviluppati su C_0 sulle varianti di DefaultJobManager

Test eseguiti	C_0	C_1	C_2	C_3	C_4
Test BB/CF/MT principali	P	P	F	F	F
Test IT load	P	P	P	P	P
Test LLM	P	P	P	P	P
Test RND	P	P	P	P	P
Test ES	N/A	N/A	N/A	N/A	N/A
Esito complessivo	P(45/45)	P(45/45)	F(1)	F(1)	F(1)

Legenda: P = nessuna failure/error; F = almeno un fallimento o errore; N/A = suite non disponibile.

```

1  /* C0: comportamento originale caratterizzato dal test */
2  context.getData().putAll(jobData);
3
4  /* C2-C3: gestione difensiva introdotta dal refactoring */
5  if (jobData != null && !jobData.isEmpty()) {
6      context.getData().putAll(jobData);
7  }
8
9  /* C4: gestione esplicita del caso null con mappa vuota */
10 if (jobData != null && !jobData.isEmpty()) {
11     context.getData().putAll(jobData);
12 } else if (jobData != null && jobData.isEmpty()) {
13     // nothing to add
14 } else if (jobData == null) {
15     context.getData().putAll(Map.of());
16 }

```

Listing 2: Differenza comportamentale nella gestione di jobData

Table 9: Confronto dei test automatici generati su C_0 e sulle varianti di DefaultJobManager

Classe	Suite	Test run	Line Cov.	Mutation Score	Δ Cov.	Δ Mut.
C_0	RND	8	0.00%	0%	baseline	baseline
C_0	LLM	12	35.50%	43%	baseline	baseline
C_0	ES	N/A	N/A	N/A	N/A	N/A
C_1	RND	7	0.00%	0%	0.00%	0%
C_1	LLM	10	30.34%	39%	-5.16%	-4%
C_1	ES	N/A	N/A	N/A	N/A	N/A
C_2	RND	7	0.00%	0%	0.00%	0%
C_2	LLM	9	32.07%	42%	-3.43%	-1%
C_2	ES	N/A	N/A	N/A	N/A	N/A
C_3	RND	7	0.00%	0%	0.00%	0%
C_3	LLM	10	31.69%	41%	-3.81%	-2%
C_3	ES	N/A	N/A	N/A	N/A	N/A
C_4	RND	7	0.00%	0%	0.00%	0%
C_4	LLM	11	31.18%	35%	-4.32%	-8%
C_4	ES	N/A	N/A	N/A	N/A	N/A

Table 10: Dettaglio delle metriche JaCoCo e PIT per le suite LLM generate su DefaultJobManager

Classe	Test run	Line Cov.	Instr. Cov.	Branch Cov.	Mutation Score	Test Strength
C_0	12	35.50%	33.57%	50.00%	43%	88%
C_1	10	30.34%	28.86%	35.00%	39%	92%
C_2	9	32.07%	31.95%	38.64%	42%	90%
C_3	10	31.69%	30.36%	36.36%	41%	90%
C_4	11	31.18%	29.39%	34.00%	35%	86%

Table 11: Issue statiche considerate nelle suite LLM generate per DefaultJobManager

Classe	Issue considerate	Naming	Duplicati	Altro
C_0	15	0	0	15
C_1	18	10	1	7
C_2	1	0	1	0
C_3	11	10	1	0
C_4	11	11	0	0

Table 12: Rappresentanti concreti per il metodo authenticate

Parametro	Classe	Rappresentante
domain	Stringa piena	"Master"
domain	Stringa vuota	""
domain	Nulla	null
authentication	Token completo	UsernamePasswordAuthenticationToken(username, password)
authentication	Token nullo	null
principal	Presente	"test-user"
principal	Nulla	null
credentials	Presenti	"secret"
credentials	Nulle	null
isAuthenticated	Non autenticato	false

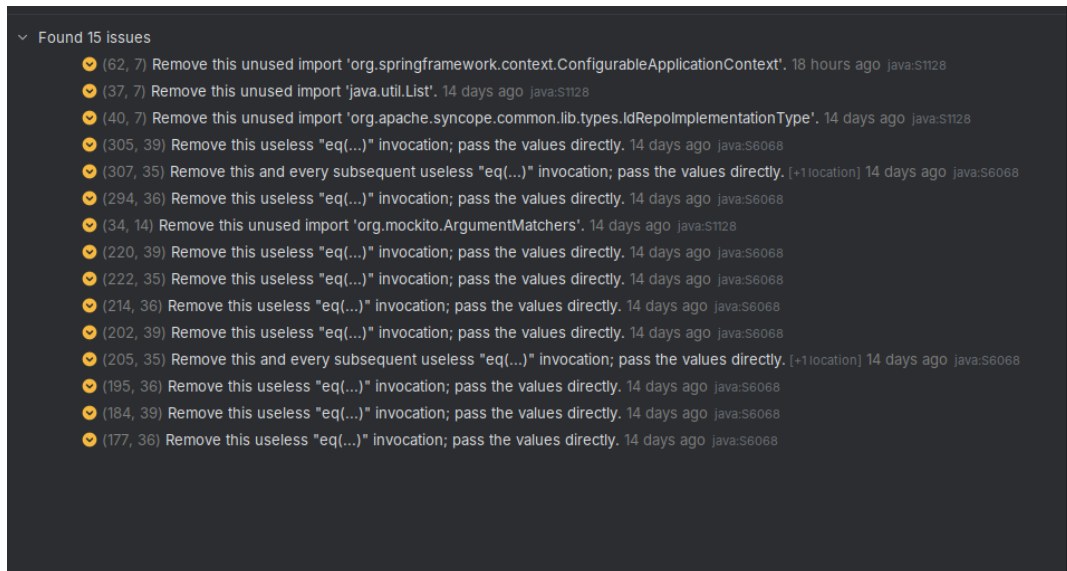
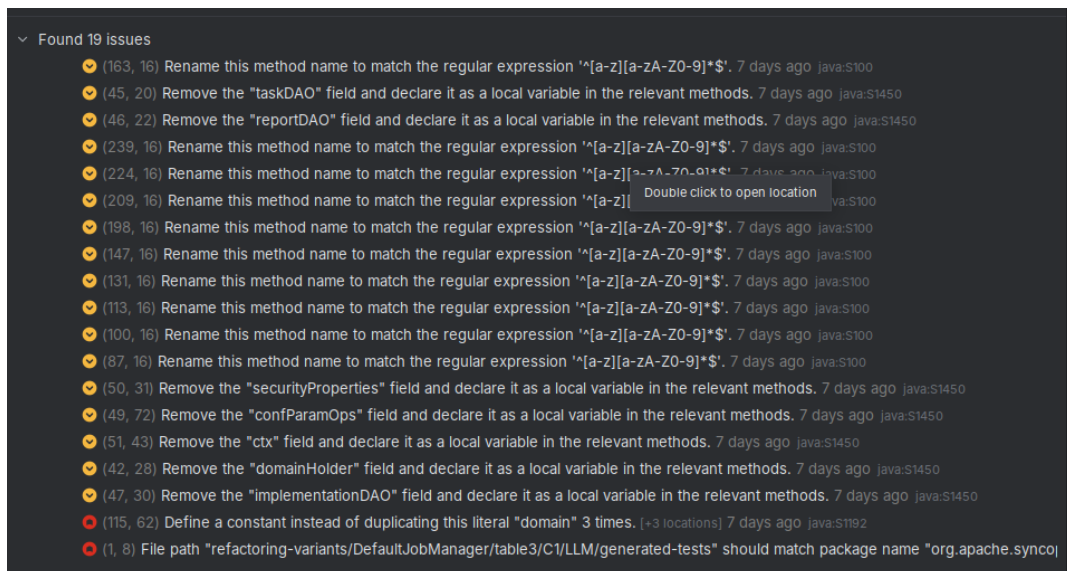
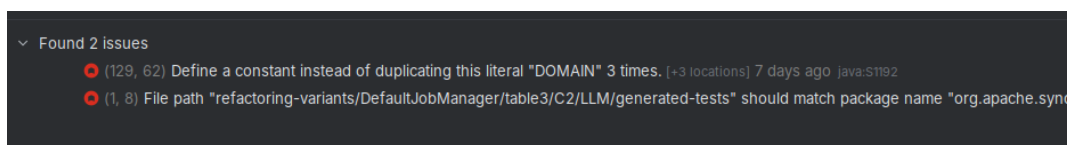
Figure 21: Issue statiche rilevate nella suite LLM generata su C_0 .Figure 22: Issue statiche rilevate nella suite LLM generata su C_1 .Figure 23: Issue statiche rilevate nella suite LLM generata su C_2 .

Table 13: Matrice dei test frame manuali per il metodo authenticate

ID	Variazione	Stato	Oracolo
A-T1	Caso nominale; domain pieno; token completo	Valido	A-O1
A-T2	domain=""; token completo	Limite	A-O1
A-T3	domain=null; token completo	Limite	A-O1
A-T4	authentication=null	Invalido	A-O2
A-T5	Token con credentials=null	Invalido	A-O2
A-T6	Token con principal=null	Invalido	A-O3

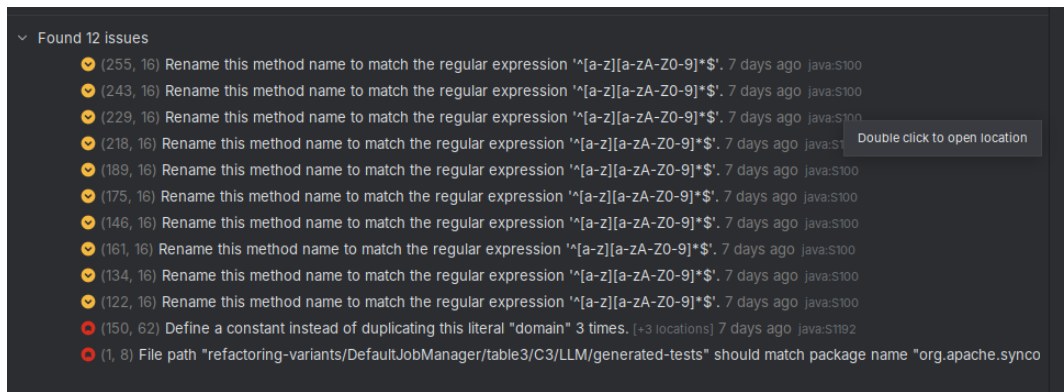
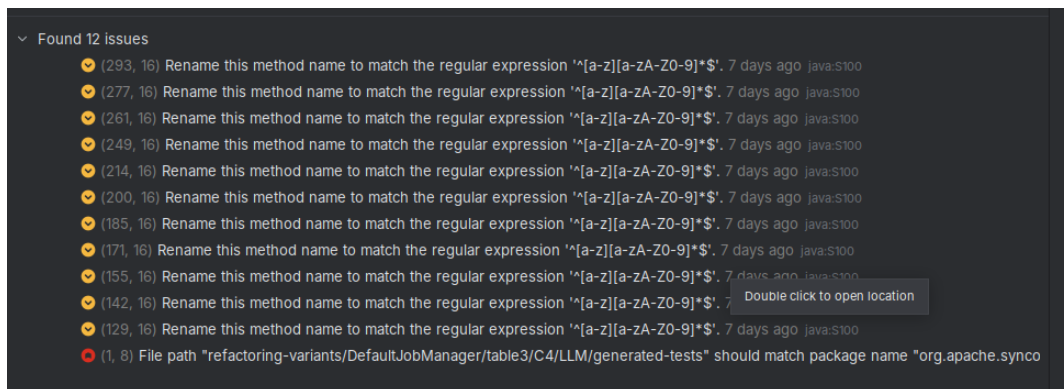
Figure 24: Issue statiche rilevate nella suite LLM generata su C_3 .Figure 25: Issue statiche rilevate nella suite LLM generata su C_4 .

Table 14: Legenda degli oracoli per il metodo authenticate

Codice	Comportamento atteso
A-O1	Autenticazione positiva; utente valorizzato; esito autenticato; delegationKey nulla
A-O2	Oracolo iniziale: rifiuto controllato dell'input assento o incompleto
A-O3	Risultato di autenticazione restituito; utente nullo; stato autenticato nullo; delegationKey nulla

Table 15: Rappresentanti concreti per il metodo getAuthorities

Parametro	Classe	Rappresentante
username	Stringa piena	"test-user"
username	Stringa vuota	""
username	Nulla	null
delegationKey	Nulla / assente	null
delegationKey	Stringa piena	"delegation-key"
delegationKey	Stringa vuota	""

org.apache.syncope.core.spring.security

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
AuthDataAccessor		24%		28%	71 83	155 219	38 46	0 1
DefaultPasswordGenerator		0%		0%	40 40	75 75	18 18	1 1
DefaultEncryptor		0%		0%	29 29	79 79	9 9	1 1
UsernamePasswordAuthenticationProvider		0%		0%	18 18	68 68	8 8	1 1
AuthContextUtils		0%		0%	25 25	62 62	18 18	1 1

Figure 26: JaCoCo: overview della coverage su AuthDataAccessor con la sola suite Black-Box.

Table 16: Matrice dei test frame manuali per il metodo `getAuthorities`

ID	Variazione	Stato	Oracolo
G-T1	Caso nominale; username pieno; nessuna delega	Valido	G-O1
G-T2	username pieno; delegationKey piena	Valido	G-O2
G-T3	username pieno; delegationKey=""	Invalido	G-O3
G-T4	username=""; nessuna delega	Invalido	G-O4
G-T5	username=null; nessuna delega	Invalido	G-O4

Table 17: Legenda degli oracoli per il metodo `getAuthorities`

Codice	Comportamento atteso
G-O1	Nessuna eccezione; set di authorities non nullo per richiesta diretta
G-O2	Nessuna eccezione; set di authorities non nullo per richiesta delegata
G-O3	Oracolo iniziale: rifiuto controllato; nessun set valido di authorities delegate
G-O4	Oracolo iniziale: rifiuto controllato; nessun set valido di authorities dirette

org.apache.syncope.core.spring.security

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
AuthDataAccessor		28%		36%	6483	145219	3746	01
DefaultPasswordGenerator		0%		0%	4040	7575	1818	11
DefaultEncryptor		0%		0%	2929	7979	99	11
UsernamePasswordAuthenticationProvider		0%		0%	1818	6868	88	11
AuthContextUtils		0%		0%	2525	6262	1818	11
JWTAuthentication		0%		0%	1717	4848	1313	11

Figure 27: JaCoCo: overview della coverage su `AuthDataAccessor` dopo i raffinamenti coverage-aware.

- lambda\$getUserNames\$0(String) - lambda\$getUserNames\$1(String) - removeExpired(String) - lambda\$getUserAuthorities\$2(Map, String) - lambda\$getUserAuthorities\$3(String, JWTAuthProvider) - getAuthorities(String, String) - lambda\$audit\$0(String, String) - lambda\$authenticate\$2(String, String) - AuthDataAccessor\$SecurityProperties, EncryptorManager, RealmSearchDAO, UserDao, GroupDAO, AnySearchDAO, AccessTokenDAO, ConfParamOps, RoleDAO, DelegationDAO, ExternalResourceDAO, ConnectorManager, AuditManager, MappingManager, List - static (...) - getAdminAuthorities() - lambda\$getUserAuthorities\$1(String) - lambda\$getUserAuthorities\$0(String)	 0%  0%  0%  0%  0%  94%  0%  0%  100%  100%  100%  100%  100%	Total 817 of 1.140 28%
---	---	------------------------------

Figure 28: JaCoCo: dettaglio della coverage per metodo dopo i raffinamenti coverage-aware.

```

245. public UsernamePasswordAuthResult authenticate(final String domain, final Authentication authentication) {
246.     User user = null;
247.
248.     String[] authAttrValues = confParamOps.get(
249.         domain, "authentication.attributes", new String[] { "username" }, String[].class);
250.     for (int i = 0; user == null && i < authAttrValues.length; i++) {
251.         if ("username".equals(authAttrValues[i])) {
252.             user = userDao.findByUsername(authentication.getName()).orElse(null);
253.         } else {
254.             AttrCond attrCond = new AttrCond(AttrCond.Type.EQ);
255.             attrCond.setSchema(authAttrValues[i]);
256.             attrCond.setExpression(authentication.getName());
257.             try {
258.                 List<User> users = anySearchDAO.search(SearchCond.of(attrCond), AnyTypeKind.USER);
259.                 if (users.size() == 1) {
260.                     user = users.getFirst();
261.                 } else {
262.                     LOG.warn("Search condition {} does not uniquely match a user", attrCond);
263.                 }
264.             } catch (IllegalArgumentException e) {
265.                 LOG.error("While searching user for authentication via {}", attrCond, e);
266.             }
267.         }
268.     }
269.
270.     Boolean authenticated = null;
271.     String delegationKey = null;
272.     if (user != null) {
273.         authenticated = false;
274.
275.         if (user.isSuspended() != null && user.isSuspended()) {
276.             throw new DisabledException("User " + user.getUsername() + " is suspended");
277.         }
278.
279.         String[] authStatuses = confParamOps.get(
280.             domain, "authentication.statuses", new String[] {}, String[].class);
281.         if (!ArrayUtils.contains(authStatuses, user.getStatus())) {
282.             throw new DisabledException("User " + user.getUsername() + " not allowed to authenticate");
283.         }
284.
285.         boolean userModified = false;
286.         authenticated = usernamePasswordAuthentication(user, authentication.getCredentials().toString());
287.         if (authenticated) {
288.             delegationKey = getDelegationKey(
289.                 SyncopeAuthenticationDetails.class.cast(authentication.getDetails()), user.getKey());
290.
291.             if (confParamOps.get(domain, "log.lastlogindate", true, Boolean.class)) {
292.                 user.setLastLoginDate(OffsetDateTime.now());
293.                 userModified = true;
294.             }
295.
296.             if (user.getFailedLogins() != 0) {
297.                 user.setFailedLogins(0);
298.                 userModified = true;
299.             }
300.         } else {

```

Figure 29: JaCoCo: dettaglio del metodo authenticate(...) dopo i raffinamenti coverage-aware, parte 1.


```

288.         delegationKey = getDelegationKey(
289.             SyncopeAuthenticationDetails.class.cast(authentication.getDetails()), user.getKey());
290.
291.         if (confParamOps.get(domain, "log.lastlogindate", true, Boolean.class)) {
292.             user.setLastLoginDate(OffsetDateTime.now());
293.             userModified = true;
294.         }
295.
296.         if (user.getFailedLogins() != 0) {
297.             user.setFailedLogins(0);
298.             userModified = true;
299.         }
300.         } else {
301.             user.setFailedLogins(user.getFailedLogins() + 1);
302.             userModified = true;
303.         }
304.
305.         if (userModified) {
306.             user = userDAO.save(user);
307.         }
308.     }
309.
310.     return new UsernamePasswordAuthResult(user, authenticated, delegationKey);
311. }
312.
313. protected boolean usernamePasswordAuthentication(final User user, final String password) {
314.     boolean authenticated = encryptorManager.getInstance().
315.         verify(password, user.getCipherAlgorithm(), user.getPassword());
316.     LOG.debug("{} authenticated on internal storage: {}", user.getUsername(), authenticated);
317.
318.     for (Iterator<? extends ExternalResource> itor = getPassthroughResources(user).iterator();
319.         itor.hasNext() && !authenticated;) {
320.
321.         ExternalResource resource = itor.next();
322.         String resourceKey = null;

```

Figure 30: JaCoCo: dettaglio del metodo authenticate(...) dopo i raffinamenti coverage-aware, parte 2.

```

439.
440.     @Transactional
441.     public Set<SyncopeGrantedAuthority> getAuthorities(final String username, f
442.         Set<SyncopeGrantedAuthority> authorities;
443.
444.     if (securityProperties.getAnonymousUser().equals(username)) {
445.         authorities = ANONYMOUS_AUTHORITIES;
446.     } else if (securityProperties.getAdminUser().equals(username)) {
447.         authorities = getAdminAuthorities();
448.     } else if (delegationKey != null) {
449.         Delegation delegation = delegationDAO.findById(delegationKey).
450.             orElseThrow(() -> new UsernameNotFoundException(
451.                 "Could not find delegation " + delegationKey));
452.
453.         authorities = delegation.getRoles().isEmpty()
454.             ? getUserAuthorities(delegation.getDelegating())
455.             : getDelegatedAuthorities(delegation);
456.     } else {
457.         User user = userDAO.findByUsername(username).
458.             orElseThrow(() -> new UsernameNotFoundException(
459.                 "Could not find any user with username " + username));
460.
461.         authorities = getUserAuthorities(user);
462.     }
463.
464.     return authorities;
465. }

```

Figure 31: JaCoCo: dettaglio del metodo getAuthorities(...) dopo i raffinamenti coverage-aware.

PIT Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	34% 74/219	25% 26/102	93% 26/28

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.syncope.core.spring.security	1	34% 74/219	25% 26/102	93% 26/28

- Project uses Spring, but the Arcmutate Spring plugin is not present.
- Unit 5 is on the classpath but the pitest junit 5 plugin is not installed.
- TestNG is on the classpath but the pitest TestNG plugin is not installed.

Report generated by [PIT 1.24.0 support](#)

Enhanced functionality available at [arcmutate.com](#)

Figure 32: PIT: Panoramica globale della mutation coverage sulla classe AuthDataAccessor.

231	1. replaced return value with "" for org/apache/syncope/core/spring/security/AuthDataAccessor::getDelegationKey → NO_COVERAGE
232	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getDelegationKey\$0 → NO_COVERAGE
250	1. changed conditional boundary → KILLED 2. negated conditional → KILLED 3. Changed increment from 1 to -1 → KILLED 4. negated conditional → KILLED
251	1. negated conditional → KILLED
255	1. removed call to org/apache/syncope/core/persistence/api/dao/search/AttrCond::setSchema → NO_COVERAGE
256	1. removed call to org/apache/syncope/core/persistence/api/dao/search/AttrCond::setExpression → NO_COVERAGE
259	1. negated conditional → NO_COVERAGE
272	1. negated conditional → KILLED
275	1. negated conditional → KILLED 2. negated conditional → SURVIVED Covering tests
281	1. negated conditional → KILLED
287	1. negated conditional → KILLED
291	1. negated conditional → KILLED
292	1. removed call to org/apache/syncope/core/persistence/api/entity/user/User::setLastLoginDate → NO_COVERAGE
296	1. negated conditional → KILLED
297	1. removed call to org/apache/syncope/core/persistence/api/entity/user/User::setFailedLogins → NO_COVERAGE
301	1. Replaced integer addition with subtraction → KILLED 2. removed call to org/apache/syncope/core/persistence/api/entity/user/User::setFailedLogins → KILLED
305	1. negated conditional → KILLED
310	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::authenticate → KILLED
319	1. negated conditional → NO_COVERAGE 2. negated conditional → NO_COVERAGE
325	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$usernamePasswordAuthentication\$0 → NO_COVERAGE
328	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$usernamePasswordAuthentication\$1 → NO_COVERAGE
331	1. negated conditional → NO_COVERAGE
341	1. replaced boolean return with false for org/apache/syncope/core/spring/security/AuthDataAccessor::usernamePasswordAuthentication → NO_COVERAGE 2. replaced boolean return with true for org/apache/syncope/core/spring/security/AuthDataAccessor::usernamePasswordAuthentication → NO_COVERAGE
349	1. negated conditional → NO_COVERAGE 2. removed call to java/util/stream/Stream::forEach → NO_COVERAGE 3. replaced boolean return with true for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$setPassthroughResources\$0 → NO_COVERAGE

Figure 33: PIT: Dettaglio dell'analisi dei mutanti per la run iniziale (Parte 1).

355	3. removed call to java/util/stream/Stream::forEach → NO_COVERAGE
351	3. replaced boolean return with true for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getPassthroughResources\$0 → NO_COVERAGE
360	1. negated conditional → NO_COVERAGE 2. negated conditional → NO_COVERAGE 2. removed call to java/util/stream/Stream::forEach → NO_COVERAGE 3. replaced boolean return with true for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getPassthroughResources\$2 → NO_COVERAGE
362	1. negated conditional → NO_COVERAGE
369	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::getPassthroughResources → NO_COVERAGE
373	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::getAdminAuthorities → SURVIVED Covering tests
374	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getAdminAuthorities\$0 → NO_COVERAGE
381	1. removed call to java/util/Map::forEach → NO_COVERAGE
385	1. removed call to org/apache/syncope/core/spring/security/SyncopeGrantedAuthority::addRealms → NO_COVERAGE
386	1. removed call to org/apache/syncope/core/spring/security/SyncopeGrantedAuthority::addRealms → NO_COVERAGE
390	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::buildAuthorities → NO_COVERAGE
394	1. negated conditional → KILLED
395	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::getUserAuthorities → KILLED
401	1. removed call to java/util/Collection::forEach → NO_COVERAGE
402	1. removed call to java/util/Set::forEach → NO_COVERAGE
405	2. removed call to org/apache/syncope/core/spring/security/AuthDataAccessor::populate → NO_COVERAGE 1. negated conditional → NO_COVERAGE
406	1. removed call to java/util/Set::forEach → NO_COVERAGE
409	2. removed call to org/apache/syncope/core/spring/security/AuthDataAccessor::populate → NO_COVERAGE
410	1. removed call to java/util/List::forEach → NO_COVERAGE
415	2. removed call to org/apache/syncope/core/spring/security/AuthDataAccessor::populate → NO_COVERAGE
416	1. removed call to java/util/Set::forEach → NO_COVERAGE
421	2. removed call to org/apache/syncope/core/spring/security/AuthDataAccessor::populate → NO_COVERAGE
422	1. removed call to java/util/List::forEach → NO_COVERAGE
428	2. removed call to org/apache/syncope/core/spring/security/AuthDataAccessor::populate → NO_COVERAGE
434	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::getUserAuthorities → NO_COVERAGE
435	1. removed call to java/util/Set::forEach → NO_COVERAGE
437	2. removed call to org/apache/syncope/core/spring/security/AuthDataAccessor::populate → NO_COVERAGE
444	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::getDelegatedAuthorities → NO_COVERAGE
446	1. negated conditional → KILLED
448	1. negated conditional → KILLED
450	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getAuthorities\$0 → KILLED
453	1. negated conditional → KILLED

Figure 34: PIT: Dettaglio dell'analisi dei mutanti per la run iniziale (Parte 2).

250	3. Changed increment from 1 to -1 → KILLED
251	4. negated conditional → KILLED
255	1. negated conditional → KILLED
256	1. removed call to org/apache/syncope/core/persistence/api/dao/search/AttrCond::setSchema → NO_COVERAGE
259	1. removed call to org/apache/syncope/core/persistence/api/dao/search/AttrCond::setExpression → NO_COVERAGE
272	1. negated conditional → NO_COVERAGE
275	1. negated conditional → KILLED
281	2. negated conditional → SURVIVED Covering tests
287	1. negated conditional → KILLED
291	1. negated conditional → KILLED
292	1. removed call to org/apache/syncope/core/persistence/api/entity/user/User::setLastLoginDate → NO_COVERAGE
296	1. negated conditional → KILLED
297	1. removed call to org/apache/syncope/core/persistence/api/entity/user/User::setFailedLogins → NO_COVERAGE

Figure 35: PIT: Analisi in dettaglio del mutante.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	34% 74/219	26% 27/102	96% 27/28

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.syncope.core.spring.security 1	1	34% 74/219	26% 27/102	96% 27/28

- Project uses Spring, but the Arcmutate Spring plugin is not present.
- [Unit 5 is on the classpath but the pitest junit 5 plugin is not installed.](#)
- [TestNG is on the classpath but the pitest TestNG plugin is not installed.](#)

Report generated by [PIT 1.24.0 support](#)

Enhanced functionality available at [arcmutate.com](#)

Figure 36: PIT: Analisi di dettaglio post-raffinamento (Run finale, Parte 2), con Test Strength al 96%.

org.apacne.syncope.core.spring.security

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
AuthDataAccessor		11%		1%	76 83	186 219	39 46	0 1
DefaultPasswordGenerator		0%		0%	40 40	75 75	18 18	1 1

Figure 37: JaCoCo: overview della coverage su AuthDataAccessor con la sola suite Randoop.

AuthDataAccessor

Element	Missed Instructions	Cov.
authenticate(String, Authentication)		0%
authenticate(WTAuthentication)		0%
usernamePasswordAuthentication(User, String)		0%
getAuthorities(String, String)		0%
getUserAuthorities(User)		0%
getDelegationKey(SyncopeAuthenticationDetails, String)		0%
lambda\$static\$0(String)		0%
getPassthroughResources(User)		0%

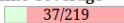
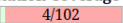
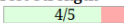
Figure 38: JaCoCo: Dettaglio dell'inefficacia di Randoop sui metodi core (0% su authenticate e delegati).

lambda\$getPassthroughResources\$2(Realm)		0%
lambda\$getPassthroughResources\$0(ExternalResource)		0%
lambda\$usernamePasswordAuthentication\$1()		0%
lambda\$usernamePasswordAuthentication\$0()		0%
lambda\$getAuthorities\$1(String)		0%
lambda\$getAuthorities\$0(String)		0%
lambda\$getUserAuthorities\$0(Map, Group)		0%
lambda\$getUserAuthorities\$3(Map, User)		0%
lambda\$getUserAdminAuthorities\$0(String)		0%
lambda\$getKeyDelegationKey\$0(String)		0%
removeExpired(String)		0%
lambda\$getUserAuthorities\$2(Map, String)		0%
lambda\$authenticate\$2(String, String)		0%
authenticate(String, String, OutEvent, Outcome, Object, Object)		91%
AuthDataAccessor\$SecurityProperties, EncryptedManager, RealmSearchDAO, UserDao, GroupDAO, AnySearchDAO, AccessTokenDAO, ConfParamOps, RoleDAO, DelegationDAO, ExternalResourceDAO, ConnectorManager, AuditManager, MappingManager, List		100%
Static (...)		100%
lambda\$authenticate\$1(String)		100%
lambda\$audit\$0(String, String)		100%
Total	1,011 of 1,140	11%

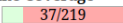
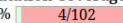
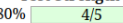
Figure 39: JaCoCo: dettaglio della coverage su getAuthorities(...) con la sola suite Randoop.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	17%  37/219	4%  4/102	80%  4/5

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.syncope.core.spring.security 1	1	17%  37/219	4%  4/102	80%  4/5

- Project uses Spring, but the Arcmutate Spring plugin is not present.
- [JUnit 5 is on the classpath but the pitest junit 5 plugin is not installed.](#)
- [TestNG is on the classpath but the pitest TestNG plugin is not installed.](#)

Report generated by [PIT 1.24.0](#) [support](#)

Enhanced functionality available at [arcmutate.com](#)

Figure 40: PIT: overview della mutation coverage su AuthDataAccessor con la sola suite Randoop.

120	1. Changed increment from 1 to -1 → NO_COVERAGE
	2. negated conditional → NO_COVERAGE
	3. changed conditional boundary → NO_COVERAGE
121	1. negated conditional → NO_COVERAGE
122	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$static\$0 → NO_COVERAGE
136	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$populate\$0 → NO_COVERAGE
207	1. negated conditional → KILLED
	1. replaced boolean return with true for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$setJTSSOPProvider\$0 → NO_COVERAGE
211	2. replaced boolean return with false for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$setJTSSOPProvider\$0 → NO_COVERAGE
	3. replaced boolean return with false for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$setJTSSOPProvider\$0 → NO_COVERAGE
212	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$setJTSSOPProvider\$1 → KILLED
217	1. negated conditional → NO_COVERAGE
218	1. replaced return value with "" for org/apache/syncope/core/spring/security/AuthDataAccessor::getDelegationKey → NO_COVERAGE
221	1. negated conditional → NO_COVERAGE
224	1. negated conditional → NO_COVERAGE
231	1. replaced return value with "" for org/apache/syncope/core/spring/security/AuthDataAccessor::getDelegationKey → NO_COVERAGE
232	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getDelegationKey\$0 → NO_COVERAGE
	1. changed conditional boundary → NO_COVERAGE
250	2. negated conditional → NO_COVERAGE
	3. Changed increment from 1 to -1 → NO_COVERAGE
	4. negated conditional → NO_COVERAGE
251	1. negated conditional → NO_COVERAGE
255	1. removed call to org/apache/syncope/core/persistence/api/dao/search/AttrCond::setSchema → NO_COVERAGE
256	1. removed call to org/apache/syncope/core/persistence/api/dao/search/AttrCond::setExpression → NO_COVERAGE
259	1. negated conditional → NO_COVERAGE
272	1. negated conditional → NO_COVERAGE
275	1. negated conditional → NO_COVERAGE
	2. negated conditional → NO_COVERAGE
281	1. negated conditional → NO_COVERAGE
287	1. negated conditional → NO_COVERAGE
291	1. negated conditional → NO_COVERAGE
292	1. removed call to org/apache/syncope/core/persistence/api/entity/user/User::setLastLoginDate → NO_COVERAGE
296	1. negated conditional → NO_COVERAGE
297	1. removed call to org/apache/syncope/core/persistence/api/entity/user/User::setFailedLogins → NO_COVERAGE
301	1. Replaced integer addition with subtraction → NO_COVERAGE
	2. removed call to org/apache/syncope/core/persistence/api/entity/user/User::setFailedLogins → NO_COVERAGE
305	1. negated conditional → NO_COVERAGE
310	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::authenticate → NO_COVERAGE
319	1. negated conditional → NO_COVERAGE
	2. negated conditional → NO_COVERAGE
325	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$usernamePasswordAuthentication\$0 → NO_COVERAGE
328	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$usernamePasswordAuthentication\$1 → NO_COVERAGE
331	1. negated conditional → NO_COVERAGE
341	1. replaced boolean return with false for org/apache/syncope/core/spring/security/AuthDataAccessor::usernamePasswordAuthentication → NO_COVERAGE
	2. replaced boolean return with true for org/apache/syncope/core/spring/security/AuthDataAccessor::usernamePasswordAuthentication → NO_COVERAGE
	1. negated conditional → NO_COVERAGE
349	2. removed call to java/util/stream/Stream::forEach → NO_COVERAGE
	3. replaced boolean return with true for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getPassthroughResources\$0 → NO_COVERAGE
351	1. negated conditional → NO_COVERAGE
	1. negated conditional → NO_COVERAGE

Figure 41: PIT: dettaglio dei mutanti su authenticate(...) con la sola suite Randoop.

435	1. removed call to org/apache/syncope/core/spring/security/AuthDataAccessor::populate → NO_COVERAGE
	2. removed call to java/util/Set::forEach → NO_COVERAGE
437	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::getDelegatedAuthorities → NO_COVERAGE
444	1. negated conditional → NO_COVERAGE
446	1. negated conditional → NO_COVERAGE
448	1. negated conditional → NO_COVERAGE
450	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getAuthorities\$0 → NO_COVERAGE
453	1. negated conditional → NO_COVERAGE
458	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$getAuthorities\$1 → NO_COVERAGE
464	1. replaced return value with Collections.emptySet for org/apache/syncope/core/spring/security/AuthDataAccessor::getAuthorities → NO_COVERAGE
472	1. negated conditional → NO_COVERAGE
474	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$authenticate\$0 → NO_COVERAGE
482	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$authenticate\$1 → NO_COVERAGE
489	1. negated conditional → NO_COVERAGE
491	1. negated conditional → NO_COVERAGE
498	1. replaced return value with "" for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$authenticate\$2 → NO_COVERAGE
501	1. negated conditional → NO_COVERAGE
507	1. negated conditional → NO_COVERAGE
511	1. negated conditional → NO_COVERAGE
517	1. replaced return value with null for org/apache/syncope/core/spring/security/AuthDataAccessor::authenticate → NO_COVERAGE
522	1. removed call to org/apache/syncope/core/persistence/api/dao/AccessTokenDAO::deleteById → KILLED
534	1. removed call to org/apache/syncope/core/provisioning/api/AuditManager::audit → KILLED
537	1. replaced return value with "" for org/apache/syncope/core/spring/security/AuthDataAccessor::lambda\$audit\$0 → SURVIVED Covering tests

Figure 42: PIT: dettaglio dei mutanti su getAuthorities(...) con la sola suite Randoop.

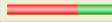
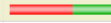
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
AuthDataAccessor		40%		47%	50	83	117	219
DefaultPasswordGenerator		0%		0%	40	40	75	75
							18	18
							1	1

Figure 43: JaCoCo: overview della coverage su AuthDataAccessor con suite manuale e suite LLM eseguite congiuntamente.

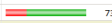
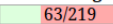
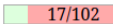
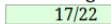
Element	Missed Instructions	Cov. %
authenticator(IWTAuthentication)		0%
usernamePasswordAuthentication(User, String)		0%
authenticator(String, Authentication)		73%
getUserAuthorities(User)		9%
lambda\$static\$0(String)		0%
getPassthroughResources(User)		0%
lambda\$getPassthroughResources\$3(Set, Realm)		0%
lambda\$getPassthroughResources\$1(Set, ExternalResource)		0%
populate(Map, String, Collection)		0%
lambda\$getUserAuthorities\$8(Map, AnyObject, String)		0%
lambda\$getUserAuthorities\$6(Map, Group, String)		0%
lambda\$getUserAuthorities\$4(Map, User, String)		0%
getDelegatedAuthorities(Delegation)		0%
lambda\$authenticate\$1(IWTAuthentication)		0%
lambda\$getUserAuthorities\$7(Map, AnyObject)		0%
lambda\$populate\$0(Map, String)		0%
lambda\$getDelegatedAuthorities\$1(Map, Role, String)		0%
lambda\$getUserAuthorities\$1(Map, Role, String)		0%
lambda\$authenticate\$0(IWTAuthentication)		0%
lambda\$getDelegatedAuthorities\$0(Map, Role)		0%
lambda\$getUserAuthorities\$0(Map, Role)		0%
lambda\$getPassthroughResources\$2(Realm)		0%
lambda\$getPassthroughResources\$0(ExternalResource)		0%
lambda\$usernamePasswordAuthentication\$1()		0%
lambda\$usernamePasswordAuthentication\$0()		0%
lambda\$getUserAuthorities\$5(Map, Group)		0%
lambda\$getUserAuthorities\$3(Map, User)		0%
lambda\$getAdminAuthorities\$0(String)		0%
lambda\$getUserAuthorities\$2(Map, String)		0%
getAuthorities(String, String)		94%
getDelegationKey(SyncopeAuthenticationDetails, String)		93%
lambda\$authenticate\$2(String, String)		0%
AuthDataAccessor\$SecurityProperties, EncryptorManager, RealmSearchDAO, UserDAO, GroupDAO, AnySearchDAO, AccessTokenDAO, ConfParamOps, RoleDAO, DelegationDAO, ExternalResourceDAO, ConnectorManager, AuditManager, MappingManager, List		100%
static {}		100%

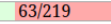
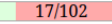
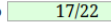
Figure 44: JaCoCo: dettaglio per metodo nella configurazione combinata manuale + LLM.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	29%  63/219	17%  17/102	77%  17/22

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
org.apache.syncope.core.spring.security	1	29%  63/219	17%  17/102	77%  17/22

- Project uses Spring, but the Arcmutate Spring plugin is not present.
- Unit 5 is on the classpath but the pitest unit 5 plugin is not installed.
- TestNG is on the classpath but the pitest TestNG plugin is not installed.

Report generated by PIT 1.24.0 [support](#)

Enhanced functionality available at [arcmutate.com](#)

Figure 45: PIT: risultati dell'esecuzione isolata della suite generata tramite LLM.

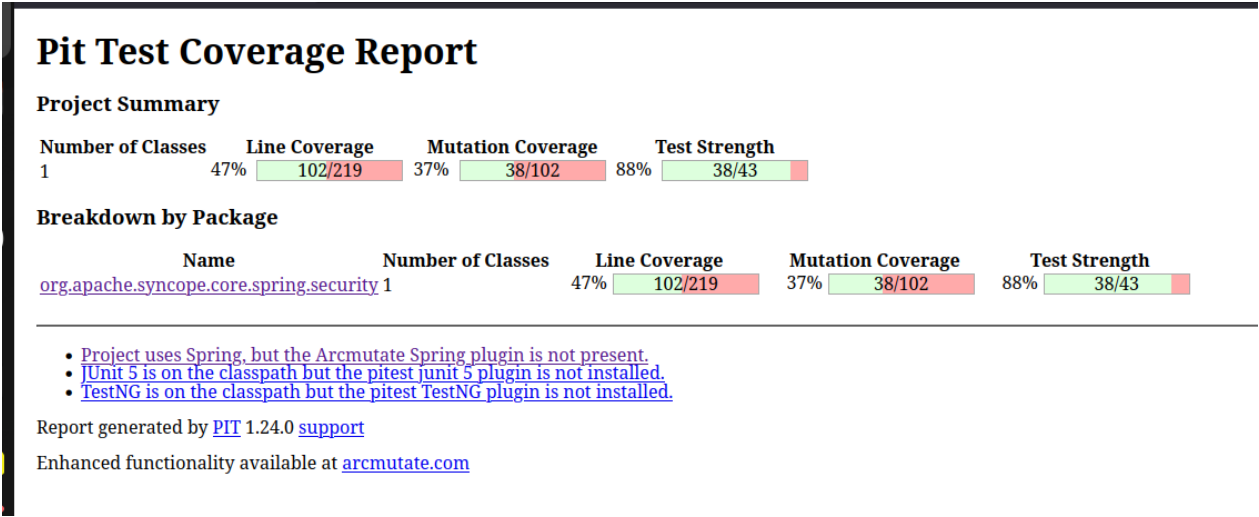


Figure 46: PIT: risultati della configurazione combinata manuale + LLM Guided Tree.


```
1 Imagine three different experts in software testing who are tasked with
2 developing comprehensive JUnit 4 test cases for the following Java class named
3 AuthDataAccessor.
4
5 The target project is Apache Syncope.
6 The Maven module is core/spring.
7 The target package is org.apache.syncope.core.spring.security.
8 The class version under test is the original variant C0.
9
10 Generate a NEW test suite specifically for the provided C0 source code.
11 Use only the provided C0 AuthDataAccessor source code as production-code reference.
12
13 The generated test file must be named AuthDataAccessorLLMGuidedTreeTest.java.
14 The generated test class must be named AuthDataAccessorLLMGuidedTreeTest.
15
16 Requirements:
17 - Use JUnit 4 only.
18 - Use Mockito and org.mockito.junit.MockitoJUnitRunner.
19 - Do not use JUnit 5 annotations or assertions.
20 - Do not generate reflection-based tests.
21 - Prefer observable behavior: return values, thrown exceptions, or interactions
22   with mocked collaborators.
23
24 Project-specific guidance:
25 - Instantiate AuthDataAccessor manually in setUp(); do not use @InjectMocks.
26 - If useful, instantiate AuthDataAccessor as a Mockito spy in setUp().
27 - Pass constructor parameters exactly in the required order.
28 - Do not pass MappingManager twice.
29 - Do not mock List<JWTSSOProvider>; use a real list.
30 - Do not import org.springframework.security.core.userdetails.User.
31 - When a user entity is needed, use
32   org.apache.syncope.core.persistence.api.entity.user.User.
33 - Do not invent JWTClaims or unavailable Syncope domain methods.
34
35 Behavioral priorities:
36 - Prioritize behaviorally relevant public or protected methods.
37 - Do not ignore authenticate if it is public or protected in the provided C0
38   source code, because authentication is central to AuthDataAccessor.
39 - Also consider authority retrieval, JWT provider lookup, delegation handling,
40   token removal, and audit delegation if safely testable.
41 - Infer normal cases, edge cases, exception cases, expected outcomes, and
42   assertions from the provided C0 source code.
43 - If a path requires complex Syncope entities, real Spring context, real
44   database access, real encryption configuration, or complex policies, prefer a
45   smaller focused test or avoid that path.
46
47 Generate fewer tests if needed, but make them more likely to compile.
```

Listing 3: Prompt principale usato per la generazione LLM Guided Tree su AuthDataAccessor

```
Expected 15 arguments but found 0
Create constructor Alt+Maiusc+Invio More actions... Alt+Invio

© org.apache.syncope.core.spring.security.AuthDataAccessor
@Contract(pure = true) ↗
public AuthDataAccessor(
    SecurityProperties securityProperties,
    EncryptorManager encryptorManager,
    RealmSearchDAO realmSearchDAO,
    UserDao userDao,
    GroupDAO groupDAO,
    AnySearchDAO anySearchDAO,
    AccessTokenDAO accessTokenDAO,
    ConfParamOps confParamOps,
    RoleDAO roleDAO,
    DelegationDAO delegationDAO,
    ExternalResourceDAO resourceDAO,
    ConnectorManager connectorManager,
    AuditManager auditManager,
    MappingManager mappingManager,
    List<JWTSSOProvider> jwtSSOProviders
)
syncope-core-spring
```

Figure 47: Errore di compilazione dovuto al disallineamento tra il costruttore legacy usato dai test EvoSuite e il costruttore corrente di `AuthDataAccessor`.

Table 18: Test sviluppati a partire da C_0 e input forniti all'LLM per la generazione delle varianti di `AuthDataAccessor`

Classe	Versione	Smell	Test BB	Test CF	Test MT	Test RND	Test ES	Test LLM
C_0	Originale	baseline	Yes	Yes	Yes	Yes	N/A	Yes
C_1	Refactored	Yes	No	No	No	No	N/A	No
C_2	Refactored	Yes	Yes	No	No	No	N/A	No
C_3	Refactored	Yes	Yes	Yes	No	No	N/A	No
C_4	Refactored	Yes	Yes	Yes	Yes	No	N/A	No

Table 19: Output osservati eseguendo i test sviluppati su C_0 sulle varianti di AuthDataAccessor

Test eseguiti	C_0	C_1	C_2	C_3	C_4
Test BB	P	P	P	P	P
Test CF	P	P	P	P	P
Test MT	P	P	P	P	P
Test LLM	P	P	P	P	P
Test RND	P	P	P	P	P
Test ES	N/A	N/A	N/A	N/A	N/A
Esito complessivo	P	P	P	P	P

Table 20: Confronto dei test automatici generati su C_0 e sulle varianti di AuthDataAccessor

Classe	Suite	Test run	Line Cov.	Mutation Score	Δ Cov.	Δ Mut.
C_0	RND	1653	15.07%	4%	baseline	baseline
C_0	LLM	10	41.55%	19%	baseline	baseline
C_0	ES	N/A	N/A	N/A	N/A	N/A
C_1	RND	1022	14.73%	4%	-0.34%	0%
C_1	LLM	8	27.68%	18%	-13.87%	-1%
C_1	ES	N/A	N/A	N/A	N/A	N/A
C_2	RND	983	15.07%	4%	0.00%	0%
C_2	LLM	11	36.53%	23%	-5.02%	+4%
C_2	ES	N/A	N/A	N/A	N/A	N/A
C_3	RND	1003	14.86%	4%	-0.21%	0%
C_3	LLM	9	39.19%	21%	-2.36%	+2%
C_3	ES	N/A	N/A	N/A	N/A	N/A
C_4	RND	1021	14.16%	4%	-0.91%	0%
C_4	LLM	10	26.99%	17%	-14.56%	-2%
C_4	ES	N/A	N/A	N/A	N/A	N/A

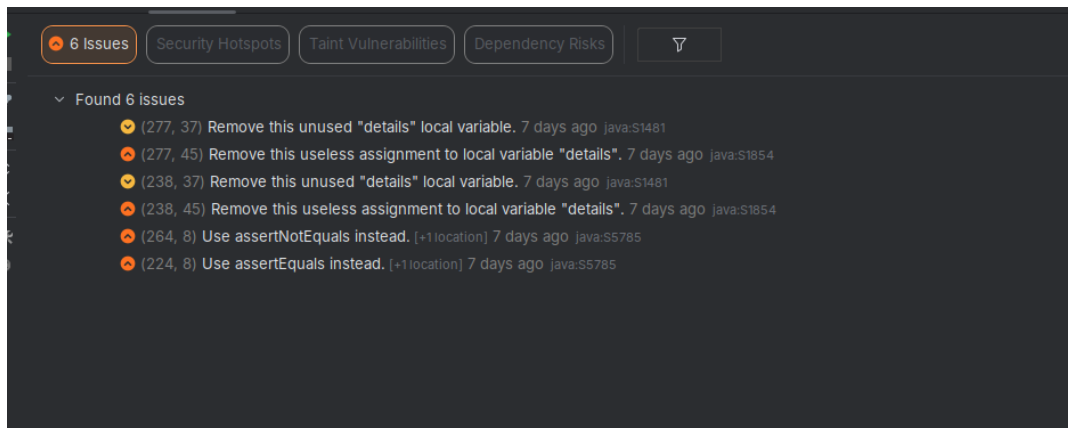
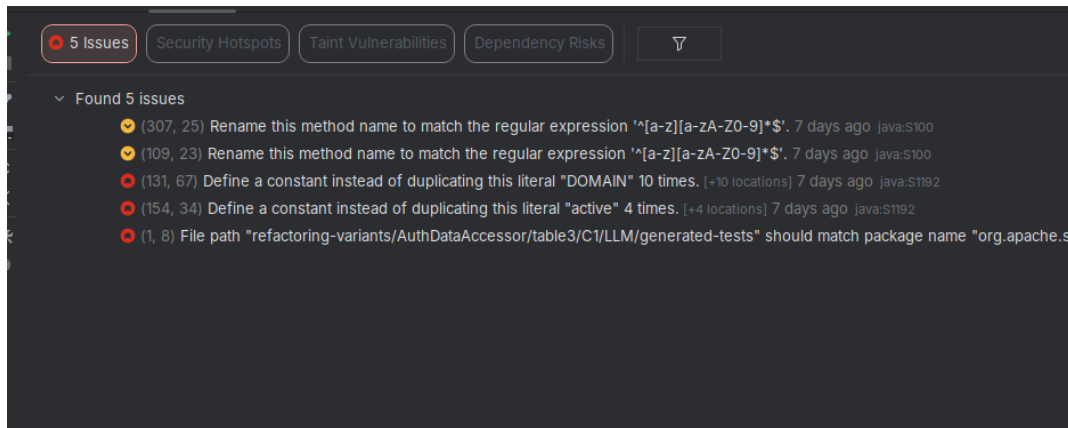
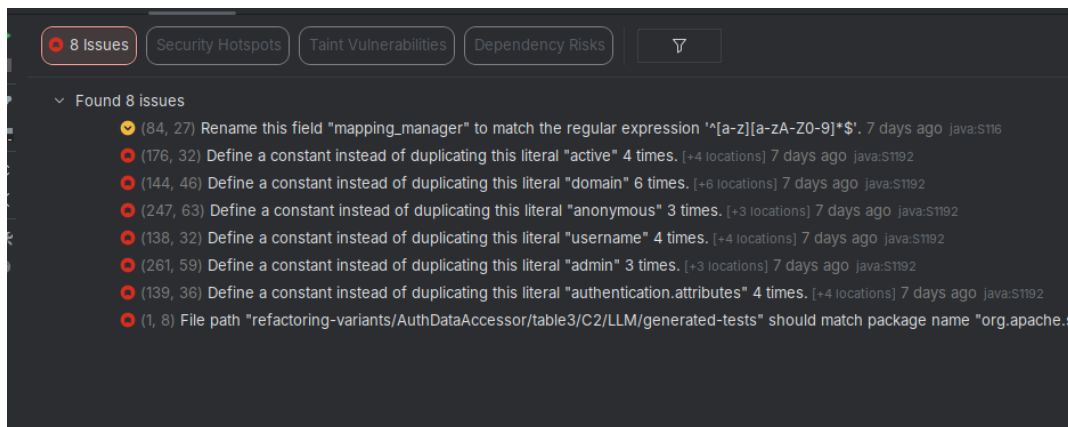
Legenda: N/A = suite non applicabile

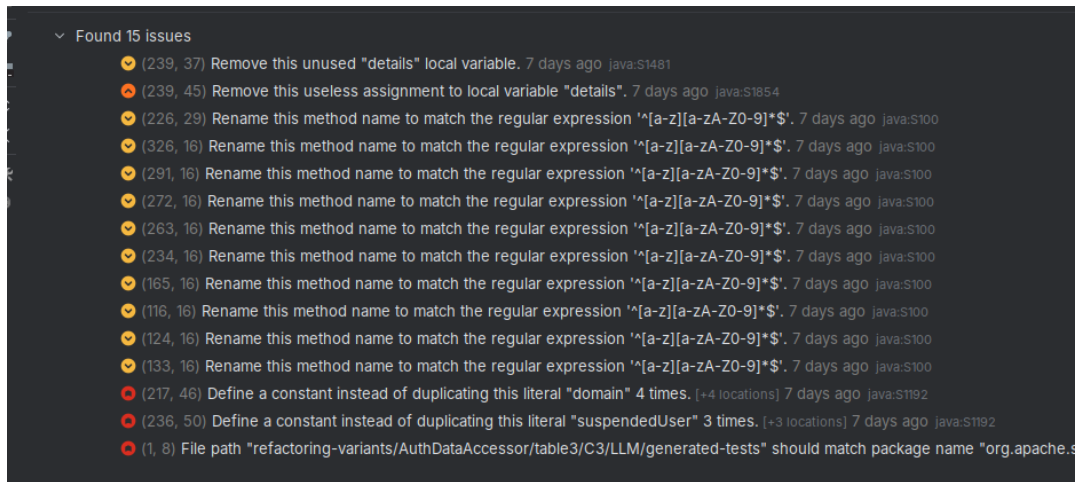
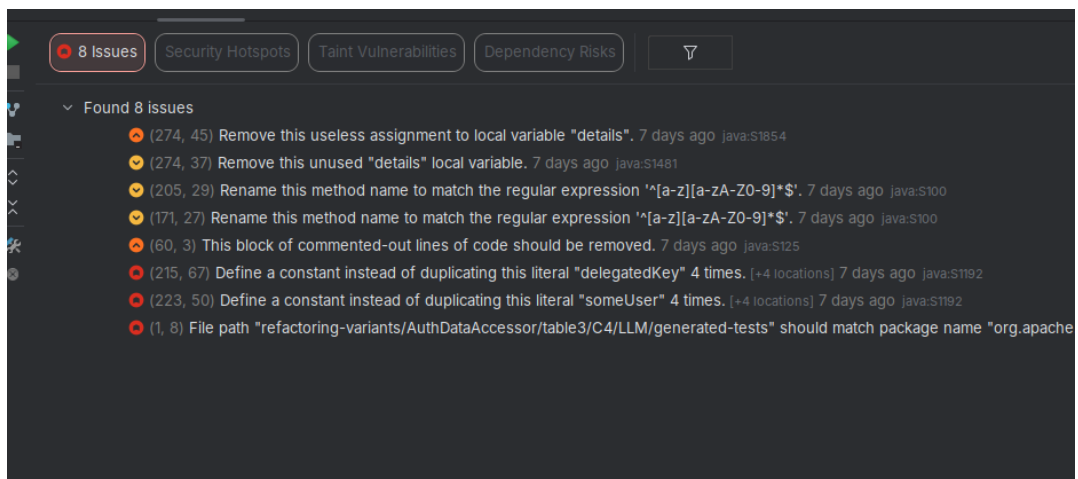
Table 21: Dettaglio delle metriche JaCoCo e PIT per le suite LLM generate su AuthDataAccessor

Classe	Test run	Line Cov.	Instr. Cov.	Branch Cov.	Mutation Score	Test Strength
C_0	10	41.55%	35.44%	33.78%	19%	46%
C_1	8	27.68%	22.43%	14.86%	18%	86%
C_2	11	36.53%	30.70%	33.78%	23%	77%
C_3	9	39.19%	33.80%	27.03%	21%	58%
C_4	10	26.99%	23.81%	21.62%	17%	75%

Table 22: Issue statiche considerate nelle suite LLM generate per AuthDataAccessor

Classe	Issue considerate	Naming	Duplicati	Altro
C_0	6	0	0	6
C_1	4	2	2	0
C_2	7	1	6	0
C_3	14	10	2	2
C_4	7	2	2	3

Figure 48: Issue statiche rilevate nella suite LLM generata per C_0 .Figure 49: Issue statiche rilevate nella suite LLM generata per C_1 .Figure 50: Issue statiche rilevate nella suite LLM generata per C_2 .

Figure 51: Issue statiche rilevate nella suite LLM generata per C_3 .Figure 52: Issue statiche rilevate nella suite LLM generata per C_4 .